ỦY BAN NHÂN DÂN TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

TÊN SINH VIÊN – MÃ SINH VIÊN THỰC HIỆN ĐỒ ÁN

Quách Hoàng Kha	3122410167
Tô Hoàng Nhật Lâm	3124410183
Nguyễn Phúc Tài	3124410308
Trần Quỳnh Ngọc	3124560062
Nguyễn Đinh Ý Vy	3124560100

TÌM HIỂU VÀ TRIỂN KHAI ỨNG DỤNG WEB TRÊN VERCEL

ĐỒ ÁN CUỐI KỲ
MÔN HỌC QUẢN TRỊ MẠNG

NGÀNH: CÔNG NGHỆ THÔNG TIN

TRÌNH ĐỘ ĐÀO TẠO: ĐẠI HỌC

TP. HCM, ngày 29 tháng 10 năm 2026

ỦY BAN NHÂN DÂN TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN

TÊN SINH VIÊN – MÃ SINH VIÊN THỰC HIỆN ĐỒ ÁN

Quách Hoàng Kha	3122410167
Tô Hoàng Nhật Lâm	3124410183
Nguyễn Phúc Tài	3124410308
Trần Quỳnh Ngọc	3124560062
Nguyễn Đinh Ý Vy	3124560100

TÌM HIỂU VÀ TRIỂN KHAI ỨNG DỤNG WEB TRÊN VERCEL

ĐỒ ÁN CUỐI KỲ
MÔN HỌC QUẢN TRỊ MẠNG

NGÀNH: CÔNG NGHỆ THÔNG TIN

TRÌNH ĐỘ ĐÀO TẠO: ĐẠI HỌC

NGƯỜI HƯỚNG DẪN: THS LƯƠNG MINH HUẤN

TP. HCM, ngày 29 tháng 10 năm 2025

LỜI CAM ĐOAN

Chúng em xin cam đoan báo cáo đồ án với đề tài "Tìm hiểu và triển khai ứng dụng web trên Vercel" là công trình nghiên cứu và thực nghiệm được thực hiện độc lập bởi nhóm dưới sự hướng dẫn trực tiếp của Thầy Lương Minh Huấn.

Chúng em xin cam kết:

1. Tính trung thực của dữ liệu: Toàn bộ nội dung phân tích, sơ đồ kiến trúc, cấu hình hệ thống cũng như các bằng chứng thực nghiệm (bao gồm log build, phản hồi HTTP và hình ảnh Dashboard từ các dịch vụ Vercel, Render, MongoDB Atlas, Brevo, Cloudinary) được trình bày trong báo cáo là hoàn toàn trung thực, khách quan và do nhóm tự tổng hợp, ghi nhận trực tiếp trong quá trình triển khai dự án Shrimple.

2. Tính chính xác của trích dẫn: Các thông tin, số liệu, tài liệu tham khảo và mã nguồn mở từ bên ngoài đều đã được nhóm khai báo, trích dẫn nguồn gốc rõ ràng và hợp lệ.

3. Tính độc lập: Báo cáo này không sao chép bất kỳ công trình nghiên cứu hay đồ án của cá nhân/nhóm nào khác và chưa từng được sử dụng để nộp lấy bằng cấp tại bất kỳ cơ sở đào tạo nào.

Nếu có bất kỳ sự sai phạm nào về tính trung thực và quy định liêm chính học thuật trong báo cáo này, nhóm chúng em xin chịu hoàn toàn trách nhiệm trước Thầy và Nhà trường.

TP. HCM, ngày 29 tháng 10 năm 2026

Đại diện nhóm sinh viên thực hiện

Quách Hoàng Kha

(Cùng các thành viên: Tô Hoàng Nhật Lâm, Nguyễn Phúc Tài, Trần Quỳnh Ngọc, Nguyễn Đinh Ý Vy)

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin bày tỏ lòng biết ơn sâu sắc đến cha mẹ đã luôn động viên, chăm lo và tạo điều kiện thuận lợi nhất cho chúng em trong suốt quá trình học tập cũng như thực hiện đồ án này.

Tiếp theo, chúng em xin gửi lời cảm ơn chân thành đến Thầy Lương Minh Huấn - giảng viên bộ môn Quản Trị Mạng, khoa Công nghệ thông tin, trường Đại học Sài Gòn, đã trang bị cho chúng em những kiến thức và kỹ năng cơ bản cần thiết để có thể hoàn thành đồ án này.

Tuy nhiên, với vốn kiến thức cũng như kinh nghiệm còn khiêm tốn và mặc dù nhóm đã cố gắng hoàn thành đồ án với tất cả nỗ lực của từng thành viên trong nhóm, nhưng đồ án này chắc chắn không tránh khỏi những thiếu sót nhất định, chúng em rất mong nhận được sự quan tâm, cảm thông, chia sẻ và góp ý chỉ bảo của Thầy để đồ án của chúng em được đầy đủ và hoàn chỉnh hơn.

Xin kính chúc Thầy dồi dào sức khỏe và hạnh phúc để tiếp tục thực hiện sứ mệnh cao đẹp của mình là truyền đạt kiến thức cho thế hệ mai sau.

Chúng em xin chân thành cảm ơn.

TP. HCM, ngày 29 tháng 10 năm 2026

MỤC LỤC

Trang bìa phụ ..... **i**

LỜI CAM ĐOAN ..... **ii**

LỜI CẢM ƠN ..... **iii**

MỤC LỤC ..... **iv**

DANH MỤC HÌNH ẢNH VÀ BẢNG BIỂU ..... **vii**

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI **1**

1.1 Bối cảnh và vấn đề nghiên cứu 1

1.2 Mục tiêu và câu hỏi nghiên cứu 1

1.3 Phạm vi và phương pháp thực hiện 1

1.4 Tổng quan chức năng và đối tượng sử dụng 1

1.5 Cách tổ chức bằng chứng trong nghiên cứu 2

1.6 Tiêu chí đánh giá 2

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT **4**

2.1 Vercel và chuỗi triển khai frontend 4

2.1.1 Vòng đời bản build và khả năng tái lập 4

2.1.2 Vai trò của nhánh và môi trường 5

2.2 Định tuyến SPA và reverse proxy 5

2.2.1 Phân biệt định tuyến máy chủ và điều hướng trình duyệt 5

2.2.2 Phân phối tài nguyên và nguyên tắc cache 6

2.3 Biến môi trường và phạm vi công khai 6

2.4 Tên miền HTTPS và CORS 6

2.5 Backend dữ liệu email và media 7

2.5.1 Kết nối thời gian thực và trạng thái sau gián đoạn 7

2.5.2 Tổ chức dữ liệu theo nghiệp vụ 8

2.5.3 Trạng thái gửi thư và trạng thái tài nguyên 8

2.6 Giám sát và khôi phục phiên bản 8

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG 10
3.1 Kiến trúc tổng thể 10
3.2 Tổ chức mã nguồn và đơn vị triển khai 11
3.3 Ranh giới trách nhiệm giữa các thành phần 12
3.4 Các nhóm dữ liệu được quan sát 12
3.5 Thiết kế định tuyến và cấu hình Vercel 13
3.5.1 Đối chiếu các đường kết nối khi đổi môi trường 14
3.6 Phân tách cấu hình frontend và backend 15
3.7 Quy trình quản lý cập nhật 15
3.8 Luồng xác thực email và các điểm đối chiếu 16
3.9 Luồng media từ upload đến hiển thị 17
CHƯƠNG 4: TRIỂN KHAI VÀ KẾT QUẢ QUAN SÁT 19
4.1 Thiết lập frontend và liên kết GitHub 19
4.1.1 Trình tự triển khai và điều kiện chuyển bước 19
4.2 Chuẩn bị bản build và cấu hình môi trường 21
4.2.1 Kiểm tra hiệu lực của cấu hình môi trường 22
4.3 Bản Production và khả năng truy vết phiên bản 22
4.3.1 Lịch sử phát hành và quan hệ giữa các mốc phiên bản 23
4.4 Phân tích log build và tài nguyên đầu ra 24
4.4.1 Phân tích cảnh báo và ưu tiên xử lý 25
4.4.2 Đối chiếu tài nguyên được phát hành 26
4.5 Tên miền và dữ liệu quan sát vận hành 26
4.5.1 Phân bố CDN và giới hạn diễn giải số liệu 27
4.6 Giao diện ứng dụng và kết quả truy cập HTTP 28
4.6.1 Đăng ký và khôi phục mật khẩu trên các URL con 30
4.6.2 Cách tái hiện kiểm tra HTTP và đọc phản hồi 31
4.7 Backend Render và cấu hình dịch vụ 32
4.7.1 Đối chiếu cấu hình build với tiến trình chạy 35
4.8 Cơ sở dữ liệu trên MongoDB Atlas 35

4.8.1 Trình tự khoanh vùng lỗi kết nối dữ liệu 36
4.9 Email giao dịch qua Brevo 36
4.9.1 Đối chiếu cấu hình email với một lần yêu cầu OTP 37
4.10 Tài nguyên media trên Cloudinary 37
4.10.1 Đối chiếu tài nguyên và xử lý các trường hợp không hoàn tất 39
4.11 Thiết kế kịch bản kiểm thử và cách ghi nhận 39
4.11.1 Phương pháp đo hiệu năng trong giai đoạn tiếp theo 40
4.12 Tổng hợp mức độ kiểm chứng 41
CHƯƠNG 5: ĐÁNH GIÁ KẾT QUẢ VÀ KẾT LUẬN 42
5.1 Kết quả đạt được 42
5.2 Phân tích kết quả theo các lớp vận hành 42
5.3 Hạn chế và các vấn đề cần hoàn thiện 43
5.4 Định hướng cải tiến và quy trình xử lý sự cố 43
5.5 Quản lý rủi ro cấu hình và dịch vụ 44
5.5.1 Quy trình phản ứng khi có sự cố 45
5.6 Thứ tự ưu tiên hoàn thiện 45
5.7 Kết luận 46
BẢNG PHÂN CHIA CÔNG VIỆC 47
TÀI LIỆU THAM KHẢO 48

DANH MỤC HÌNH ẢNH VÀ BẢNG BIỂU

Hình ảnh

Hình 3.1. Kiến trúc triển khai ứng dụng Shrimple 10
Hình 3.2. Cấu trúc thư mục dự án 11
Hình 3.3. Nội dung tệp cấu hình client/vercel.json 13
Hình 3.4. Phân luồng URL giao diện REST API và Socket.IO 14
Hình 3.5. Quy trình đề xuất để quản lý cập nhật frontend 16
Hình 3.6. Luồng OTP và các điểm đối chiếu kết quả gửi thư 17
Hình 3.7. Luồng đối chiếu upload và hiển thị tài nguyên media 18
Hình 4.1. Repository liên kết và thư mục gốc của frontend 20
Hình 4.2. Framework Vite và cấu hình riêng của bản Production 21
Hình 4.3. Các lệnh kiểm tra và chuẩn bị bản build frontend 21
Hình 4.4. Các biến môi trường frontend trong Production và Preview 22
Hình 4.5. Bản Production Vercel tại commit 214717b 23
Hình 4.6. Lịch sử deployment Vercel và các mốc commit 24
Hình 4.7. Log tạo bundle và hoàn tất deployment Vercel 25
Hình 4.8. Danh sách Static Assets của deployment frontend 26
Hình 4.9. Tên miền Production có cấu hình hợp lệ 27
Hình 4.10. Dữ liệu Observability theo môi trường và khoảng thời gian 27
Hình 4.11. Phân bố phục vụ CDN trong khung Last 12 hours 28
Hình 4.12. Giao diện đăng nhập của Shrimple trên Vercel 29
Hình 4.13. Bảng tin Shrimple với bài viết và tài nguyên media 30
Hình 4.14. Trò chuyện nhóm và nội dung chia sẻ vị trí 30
Hình 4.15. Giao diện đăng ký và khôi phục mật khẩu tại URL con 31
Hình 4.16. Lệnh cURL 32
Hình 4.17. Web service Render và lịch sử triển khai backend 33
Hình 4.18. Giao diện lệnh build và start 33
Hình 4.19. Các nhóm biến môi trường backend trên Render 34
Hình 4.20. Log backend kết nối dữ liệu và khởi động thành công 34
Hình 4.21. MongoDB Atlas và các collection của cơ sở dữ liệu test 36
Hình 4.22. Khóa API Brevo của tích hợp backend ở trạng thái Active 37

Hình 4.23. Thư viện ảnh và video trong Cloudinary.....38
Hình 4.24. Mức sử dụng tài nguyên Cloudinary trong khung thời gian hiển thị 38

Bảng biểu

Bảng 1.1. Tiêu chí đánh giá và loại bằng chứng 2
Bảng 3.1. Vai trò của các thành phần trong hệ thống 10
Bảng 3.2. Nhóm collection theo chức năng trong ảnh MongoDB Atlas.....12
Bảng 3.3. Các đường kết nối cần đối chiếu giữa môi trường.....14
Bảng 3.4. Phạm vi sử dụng các biến cấu hình chính.....15
Bảng 4.1. Cấu hình triển khai frontend trên Vercel 19
Bảng 4.2. Các chỉ tiêu build được ghi nhận 25
Bảng 4.3. Kết quả HTTP không có phiên đăng nhập ngày 16 tháng 10 năm 2026 29
Bảng 4.4. Kế hoạch kiểm thử để bổ sung nghiệm thu chức năng 39
Bảng 4.5. Mức độ kiểm chứng sau khi bổ sung ảnh dịch vụ 41
Bảng 5.1. Định hướng khoanh vùng sự cố theo từng lớp.....43
Bảng 5.2. Rủi ro vận hành và biện pháp kiểm soát đề xuất 44
Bảng 5.3. Phân công nhiệm vụ của các thành viên 47

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1 Bối cảnh và vấn đề nghiên cứu

Triển khai web đòi hỏi người quản trị xác định đúng nơi xử lý yêu cầu, cấu hình kết nối dịch vụ và phiên bản đang phục vụ. Đề tài khảo sát frontend Shrimple trên Vercel, kết hợp backend Render, dữ liệu MongoDB Atlas, email Brevo và media Cloudinary. Mô hình này liên hệ kiến thức tên miền, HTTPS và định tuyến với một ứng dụng mạng xã hội kết hợp nhắn tin. [10]

1.2 Mục tiêu và câu hỏi nghiên cứu

Đề tài đối chiếu cấu hình triển khai với bản build và phản hồi thực tế, tập trung vào ba câu hỏi:

1. Cấu hình nào giúp Vercel build đúng frontend trong repository nhiều thành phần?

2. Yêu cầu tải giao diện, REST API và Socket.IO được xử lý ở đâu?

3. Bằng chứng nào xác nhận kết quả triển khai và phần nào còn cần kiểm thử?

1.3 Phạm vi và phương pháp thực hiện

Phạm vi chính gồm build, môi trường, routing, tên miền và giám sát frontend React và Vite. Các dịch vụ phía sau được phân tích để giải thích kết nối và giới hạn vận hành. Phương pháp kết hợp phân tích mã nguồn, cấu hình, tài liệu chính thức, ảnh Dashboard và kết quả HTTP đã ghi nhận.

Tập dữ liệu quan sát được tổng hợp qua các đợt thực nghiệm chính và các minh chứng bổ sung trong quá trình vận hành hệ thống. Mốc thời gian ghi nhận dữ liệu được phân định rõ với thời điểm triển khai thực tế. Mọi nhận định về thiết kế, trạng thái trên Dashboard và kết quả kiểm thử đều được phân loại, đánh giá dựa trên các bằng chứng đối chiếu tương ứng. [10]

1.4 Tổng quan chức năng và đối tượng sử dụng

Shrimple kết hợp bảng tin mạng xã hội với trao đổi trực tiếp và theo nhóm. Người sử dụng tương tác qua trình duyệt để xem nội dung, quản lý hồ sơ, kết bạn và trò chuyện. Phần xác thực tạo ranh giới giữa các trang công khai như đăng nhập, đăng ký, khôi phục mật khẩu với các trang sử dụng dữ liệu cá nhân. Từ đặc điểm đó, bài toán triển khai phải xét cả khả năng tải giao diện lẫn khả năng duy trì phiên và nhận dữ liệu đúng quyền. [10]

Các nhóm chức năng có yêu cầu hạ tầng khác nhau. Bảng tin và hồ sơ chủ yếu trao đổi dữ liệu qua REST; chat cần thêm kênh sự kiện thời gian thực; ảnh và video cần lưu trữ tài nguyên; OTP cần dịch vụ thư giao dịch. Cuộc gọi, camera và dịch thuật mở rộng yêu cầu sang thiết bị và dịch vụ xử lý bổ sung. Việc phân nhóm giúp xác định chức năng nào bị ảnh hưởng khi một thành phần gặp sự cố, thay vì xem mọi biểu hiện là lỗi giao diện. [10]

1.5 Cách tổ chức bằng chứng trong nghiên cứu

Đơn vị đối chiếu của báo cáo là một cấu hình, deployment hoặc phép thử có ngữ cảnh xác định. Với cấu hình, cần biết tệp hoặc màn hình chứa thông tin; với deployment, cần biết nền tảng, môi trường và commit; với phép thử, cần biết điều kiện xác thực, URL, thời điểm cùng kết quả. Ảnh được đặt sau đoạn giới thiệu vấn đề và được diễn giải bằng thông tin có thể đọc trực tiếp trên ảnh.

Các câu hỏi về thiết kế được trả lời bằng cấu trúc dự án và cấu hình. Các câu hỏi về trạng thái triển khai được trả lời bằng Dashboard và log. Các câu hỏi về nghiệp vụ chỉ có thể kết luận khi có chuỗi thao tác đầu vào và đầu ra tương ứng. Chẳng hạn, nhìn thấy khóa Brevo Active đủ để mô tả trạng thái khóa, còn kết luận nhận OTP cần thêm thông tin giao thư. Cách tổ chức này cho phép mở rộng kết quả mà vẫn giữ mức độ chắc chắn phù hợp.

1.6 Tiêu chí đánh giá

Bảng 1.1. Tiêu chí đánh giá và loại bằng chứng

Nhóm tiêu chí	Nội dung đánh giá	Bằng chứng phù hợp
Triển khai	Build đúng client và tạo dist; nhận diện bản Production	Thiết lập dự án, log build và commit
Định tuyến	URL con tải được; /api nhận phản hồi từ backend	Ảnh giao diện và mã HTTP kèm kiểu nội dung
Dịch vụ hỗ trợ	Backend hoạt động; các tích hợp có cấu hình và tài nguyên	Log Render; Dashboard MongoDB, Brevo, Cloudinary
Nghiệp vụ và vận hành	Xác thực, quyền, realtime và hiệu năng đạt yêu cầu	Kịch bản kiểm thử đầu cuối và số liệu đo

Tiêu chí cuối cần dữ liệu kiểm thử riêng. Trạng thái Ready, Live hoặc Active được đọc theo ý nghĩa của từng nền tảng và không đồng nghĩa mọi chức năng ứng dụng đã đạt yêu cầu.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Vercel và chuỗi triển khai frontend

Vercel hỗ trợ triển khai dự án Vite từ mã nguồn. Với ứng dụng trang đơn SPA, bước build tạo HTML, JavaScript, CSS và tài nguyên tĩnh; nền tảng phân phối các tệp này, còn React thực thi trong trình duyệt. Trong mô hình Shrimple, Vercel phục vụ frontend, trong khi tiến trình backend chạy riêng trên Render. [1][10]

Một cấu hình build cần thống nhất thư mục ứng dụng, công cụ build và thư mục đầu ra. Root Directory chỉ phạm vi chứa package frontend; Install Command cài dependency; Build Command tạo sản phẩm; Output Directory xác định tài nguyên được phát hành. Đường dẫn đầu ra được hiểu tương đối từ ứng dụng đã chọn. Vì vậy, với Root Directory là client, giá trị dist tương ứng client/dist trên cây repository. [1][10]

Liên kết Git cho phép gắn deployment với nhánh và commit. Nhánh Production cung cấp nguồn cho môi trường chính; Preview hỗ trợ đánh giá thay đổi trước khi xuất bản. Việc ghi rõ commit và thời điểm giúp truy vết lỗi khi domain Production được chuyển sang một deployment khác. [1]

2.1.1 Vòng đời bản build và khả năng tái lập

Một deployment frontend hình thành qua các bước lấy mã nguồn, chọn thư mục ứng dụng, cài dependency, chạy build và phát hành đầu ra. Mỗi bước có loại lỗi khác nhau: quyền truy cập repository ảnh hưởng việc lấy mã; lockfile ảnh hưởng cài đặt; lỗi cú pháp ảnh hưởng biên dịch; sai thư mục đầu ra ảnh hưởng tài nguyên được phục vụ. Phân tích theo vòng đời giúp xác định bước thất bại trước khi sửa cấu hình. [1]

npm ci phù hợp với quy trình cài đặt dựa trên lockfile. Theo tài liệu npm, lệnh yêu cầu lockfile có sẵn và dừng nếu khai báo dependency không khớp package.json; nó không cập nhật lockfile để tự giải quyết khác biệt. Trong dự án có client và server, lockfile phải được đối chiếu theo đúng package được triển khai. Cách làm này hỗ trợ tái lập môi trường dependency của cùng một phiên bản mã nguồn. [8]

Khả năng tái lập vẫn cần ghi nhận công cụ và môi trường chạy. Khoảng phiên bản trong package.json mô tả yêu cầu dependency, còn log và lockfile phản ánh thông tin của lần cài đặt cụ thể. Vì vậy, báo cáo phân biệt nhánh Vite 7 khai báo trong package

với Vite v7.3.0 hiển thị ở build log. Sự khác biệt cần được giải thích từ phiên bản đang khảo sát, không được sửa số liệu để chúng trông giống nhau. [10]

2.1.2 Vai trò của nhánh và môi trường

GitHub dùng repository, branch, commit và pull request để tổ chức thay đổi. Một nhánh làm việc cho phép phát triển độc lập; commit tạo mốc phiên bản; pull request cung cấp nơi so sánh trước khi hợp nhất. Khi kết nối với Vercel, các mốc này trở thành thông tin truy vết deployment và hỗ trợ kiểm tra thay đổi theo từng môi trường. [1][10]

Development, Preview và Production khác nhau về mục đích sử dụng và cấu hình được áp dụng. Một biến đúng ở Production chưa chắc đúng ở Preview vì URL và chính sách origin có thể khác. Quy trình phù hợp cần kiểm tra cả mã nguồn lẫn địa chỉ đang mở; chỉ thay nhánh trong repository mà vẫn xem domain Production có thể khiến người kiểm tra tiếp tục nhìn bản cũ. [1][10]

2.2 Định tuyến SPA và reverse proxy

SPA sử dụng bộ định tuyến phía trình duyệt để chọn màn hình. Khi người dùng truy cập trực tiếp một URL con, máy chủ vẫn cần trả điểm vào của ứng dụng để React Router xử lý. Nếu chỉ cung cấp tệp theo đường dẫn vật lý mà không có fallback phù hợp, trang đầu có thể hoạt động trong khi URL con trả 404. [1][10]

Rewrite cho phép xử lý yêu cầu bằng một tài nguyên hoặc máy chủ khác mà vẫn giữ URL người dùng truy cập. External rewrite có thể đóng vai trò reverse proxy cho REST API. Với Shrimple, nhóm /api được chuyển sang Render, còn các đường dẫn giao diện còn lại được đưa về index.html. Thứ tự quy tắc giúp yêu cầu API nhận JSON thay vì bị xử lý như HTML của SPA. [1][10]

2.2.1 Phân biệt định tuyến máy chủ và điều hướng trình duyệt

Điều hướng nội bộ sau khi React đã khởi động thường không yêu cầu máy chủ trả một tệp HTML riêng cho từng màn hình. Ngược lại, thao tác nhập URL con vào thanh địa chỉ hoặc tải lại trang bắt đầu bằng một yêu cầu HTTP mới. Đây là lý do phải kiểm tra cả điều hướng trong ứng dụng lẫn truy cập trực tiếp. Hai tình huống có thể cho kết quả khác nhau dù dùng cùng một đường dẫn hiển thị. [10]

Đối với API, tiêu chí kiểm tra còn bao gồm Content-Type. HTTP 200 kèm HTML có thể là phản hồi hợp lệ cho trang giao diện nhưng không đáp ứng một client đang chờ JSON. Một endpoint cần đăng nhập trả 401 JSON lại có thể phù hợp với kịch bản chưa xác thực. Mã trạng thái chỉ có ý nghĩa khi gắn với mục đích endpoint và điều kiện gửi request. [10]

2.2.2 Phân phối tài nguyên và nguyên tắc cache

Tệp giao diện đã build và phản hồi dữ liệu cá nhân có đặc điểm sử dụng khác nhau. Bundle JavaScript, CSS và ảnh công khai có thể được phân phối lặp lại cho nhiều người dùng; API hồ sơ hoặc tin nhắn phụ thuộc phiên và quyền. Do đó, chính sách cache cần được xác định theo loại nội dung, tránh suy rằng mọi phản hồi đi qua cùng domain đều có thể lưu dùng chung. [1][10]

External rewrite có thể chịu tác động của các header cache từ nguồn hoặc cấu hình nền tảng. Khi rà soát API cá nhân, cần xem phản hồi của đúng endpoint, thông tin xác thực và header điều khiển cache. Báo cáo xem đây là yêu cầu kiểm tra vận hành, chưa xác nhận đã kiểm toán toàn bộ endpoint. Việc frontend được phục vụ qua mạng phân phối không thay thế chính sách bảo vệ dữ liệu của backend. [1]

2.3 Biến môi trường và phạm vi công khai

Biến môi trường phân tách cấu hình theo môi trường triển khai. Với Vite, các biến được sử dụng qua import.meta.env và có tiền tố VITE_ được đưa vào mã phía trình duyệt trong bước build. Vì vậy, biến frontend chỉ chứa cấu hình có thể công khai; chuỗi kết nối dữ liệu, khóa ký token và khóa dịch vụ bí mật phải thuộc backend. [1][2]

Cập nhật biến trên Dashboard không tự sửa bundle đã phát hành. Cần tạo deployment mới và kiểm tra request thực tế của thành phần chịu ảnh hưởng. API, Socket.IO và tài nguyên media có thể có địa chỉ riêng, nên thay một biến không bảo đảm tất cả luồng kết nối cùng thay đổi. [1][2][10]

2.4 Tên miền HTTPS và CORS

Tên miền cung cấp điểm truy cập ổn định cho frontend. Vercel hỗ trợ HTTPS và quản lý chứng thư cho tên miền đáp ứng điều kiện cấu hình; tên miền tùy chỉnh cần

DNS phù hợp. Trong đề tài, địa chỉ phục vụ là https://shrimple-chat.vercel.app; bộ minh chứng chưa có tên miền riêng của nhóm. [1]

Origin gồm giao thức, host và cổng. Khi trình duyệt gọi trực tiếp sang origin khác, CORS quyết định khả năng đọc phản hồi theo chính sách server. Với request có credentials, cấu hình origin và credentials phải tương ứng. CORS không thay thế xác thực hoặc kiểm tra quyền trên API. [7]

2.5 Backend dữ liệu email và media

Render Web Service cung cấp nơi chạy tiến trình Node.js và nhận kết nối WebSocket. Socket.IO của Shrimple dùng backend để xử lý sự kiện thời gian thực và signaling cuộc gọi; WebRTC phụ trách luồng media của cuộc gọi thông thường. Những luồng dịch âm thanh bổ sung có thể đi qua backend và dịch vụ AI theo thiết kế, nên cần được đánh giá riêng. [3][10]

MongoDB tổ chức dữ liệu theo document và collection; Mongoose cung cấp lớp model cho backend. Thiết kế có thể dùng nhúng hoặc tham chiếu tùy cách truy cập. Kết nối Atlas còn phụ thuộc database user, danh sách IP được phép và TLS. Đây là trách nhiệm của backend, không phải cấu hình kết nối được phân phối cho trình duyệt. [4]

Brevo cung cấp API gửi email giao dịch, phù hợp với mã xác minh và khôi phục mật khẩu. Trạng thái khóa truy cập, kết quả API chấp nhận gửi và thư đến hộp thư là các bằng chứng ở những bước khác nhau. Cloudinary cung cấp API và SDK upload media; backend dùng thông tin xác thực để tải tài nguyên và nhận URL phục vụ. [5][6]

2.5.1 Kết nối thời gian thực và trạng thái sau gián đoạn

REST phù hợp để yêu cầu và nhận một phản hồi theo thao tác; kênh Socket.IO bổ sung cơ chế trao đổi sự kiện giữa các bên đang kết nối. Hai luồng có thể dùng chung backend nhưng khác cách khởi tạo, xác thực và xử lý mất kết nối. Một phép GET thành công vì thế chưa đủ để đánh giá kênh chat hoặc signaling cuộc gọi. [3][10]

Tài liệu Socket.IO cho biết sự kiện phát khi client đang mất kết nối có thể được giữ trong bộ đệm và gửi sau khi kết nối lại. Điều này có thể tạo đợt gửi dồn khi mạng phục hồi. Tính năng connection state recovery cần được bật và vẫn có trường hợp không

khôi phục được; ứng dụng cần có cách đồng bộ lại trạng thái. Đây là căn cứ xây dựng ca kiểm thử reconnect, không phải bằng chứng Shrimple đã bật tính năng recovery. [9]

2.5.2 Tổ chức dữ liệu theo nghiệp vụ

Collection là đơn vị tổ chức document, còn schema và cách liên kết cần xuất phát từ truy vấn ứng dụng. Dữ liệu hội thoại và tin nhắn đặt ra yêu cầu tìm theo cuộc trò chuyện và thứ tự thời gian; quan hệ bạn bè đặt ra yêu cầu kiểm soát trạng thái và người tham gia. Các ví dụ này giải thích vì sao chỉ có danh sách collection chưa đủ để đánh giá chất lượng mô hình dữ liệu hoặc index. [4]

Việc truy cập Atlas còn có hai lớp danh tính cần phân biệt: tài khoản quản trị Atlas và database user dùng để kết nối cơ sở dữ liệu. Backend cần thông tin của lớp kết nối phù hợp cùng điều kiện mạng và TLS. Khi điều tra lỗi, nên đối chiếu từng lớp này với log thay vì thay chuỗi kết nối hoặc mở quyền truy cập một cách thiếu căn cứ. [4]

2.5.3 Trạng thái gửi thư và trạng thái tài nguyên

Luồng email có các mốc từ yêu cầu nghiệp vụ, gọi dịch vụ gửi đến phát thư. Một phản hồi API được chấp nhận chưa luôn đồng nghĩa người nhận đã thấy thư; việc đánh giá cần gắn mã hoặc sự kiện gửi với hộp thư thử nghiệm. Với OTP, điều kiện xác minh còn liên quan mã hợp lệ, thời hạn và giới hạn thử. Các điều kiện này phải được ghi trong kịch bản nghiệm thu thay vì chỉ chụp biểu mẫu nhập mã. [5]

Luồng media cũng cần nhiều điểm đối chiếu: tệp đầu vào, xử lý ở backend, tài nguyên được tạo trong dịch vụ và URL được frontend sử dụng. Media Library chứng minh tài khoản có tài nguyên nhưng chưa xác định tệp đó được tạo bởi lần upload nào. Một minh chứng đầy đủ cần nối định danh hoặc URL giữa kết quả API, Cloudinary và nội dung hiển thị của Shrimple. [6]

2.6 Giám sát và khôi phục phiên bản

Build Logs giải thích quá trình lấy mã nguồn, cài dependency và build của một deployment. Observability cung cấp dữ liệu vận hành theo môi trường và khoảng thời gian; Web Analytics và Speed Insights cần được tích hợp và có dữ liệu thật trước khi

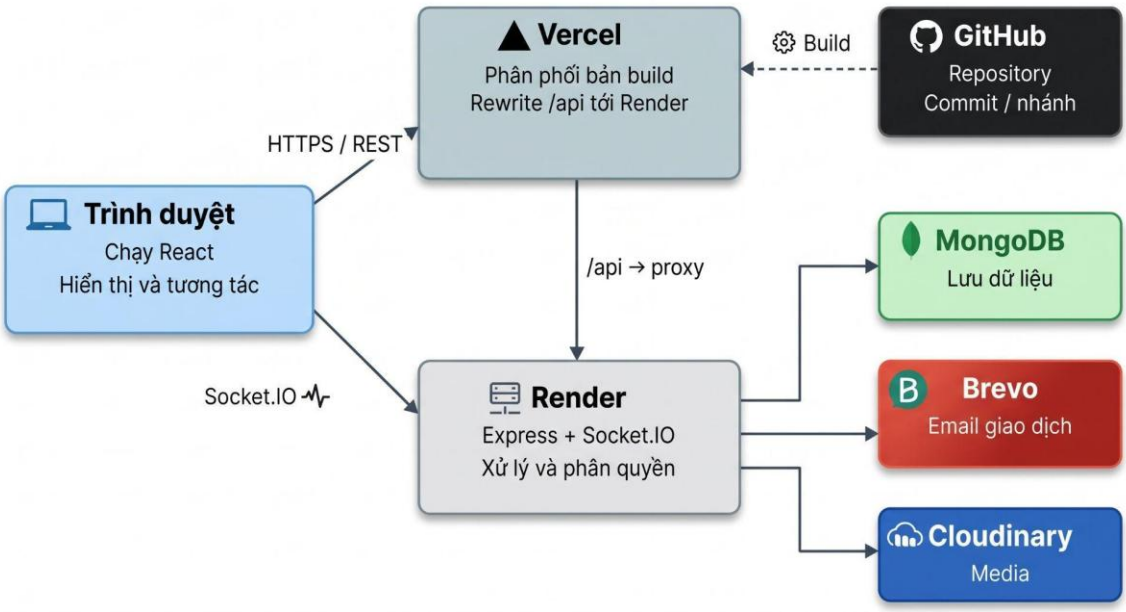
dùng để báo cáo lượt truy cập hoặc trải nghiệm. Các số hiển thị trong màn hình DEMO DATA không phải kết quả đo của ứng dụng. [1]

Instant Rollback đưa domain Production về deployment đủ điều kiện theo gói dịch vụ. Việc khôi phục frontend cần đánh giá tương thích với API và cấu trúc dữ liệu hiện tại; nó không khôi phục dữ liệu MongoDB hay đảo ngược email đã gửi. [1]

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1 Kiến trúc tổng thể

Kiến trúc Shrimple tách nơi chạy giao diện khỏi nơi xử lý nghiệp vụ. GitHub lưu mã nguồn và phiên bản; Vercel phân phối bản build; Render cung cấp API và Socket.IO; MongoDB Atlas lưu dữ liệu; Brevo và Cloudinary phục vụ email và media. Hình 3.1 mô tả luồng chính của mô hình, với REST qua /api khi client sử dụng địa chỉ cùng origin. [10]



Hình 3.1. Kiến trúc triển khai ứng dụng Shrimple

Sau khi nhận tài nguyên từ Vercel, trình duyệt chạy React và phát sinh yêu cầu nghiệp vụ. Với REST cùng origin, Vercel chuyển /api đến Render. Socket.IO có URL riêng tới Render; URL media do ứng dụng trả về có thể trỏ đến dịch vụ tài nguyên. Vì vậy, các luồng giao diện, dữ liệu và thời gian thực có điểm kiểm tra khác nhau. [10]

Bảng 3.1. Vai trò của các thành phần trong hệ thống

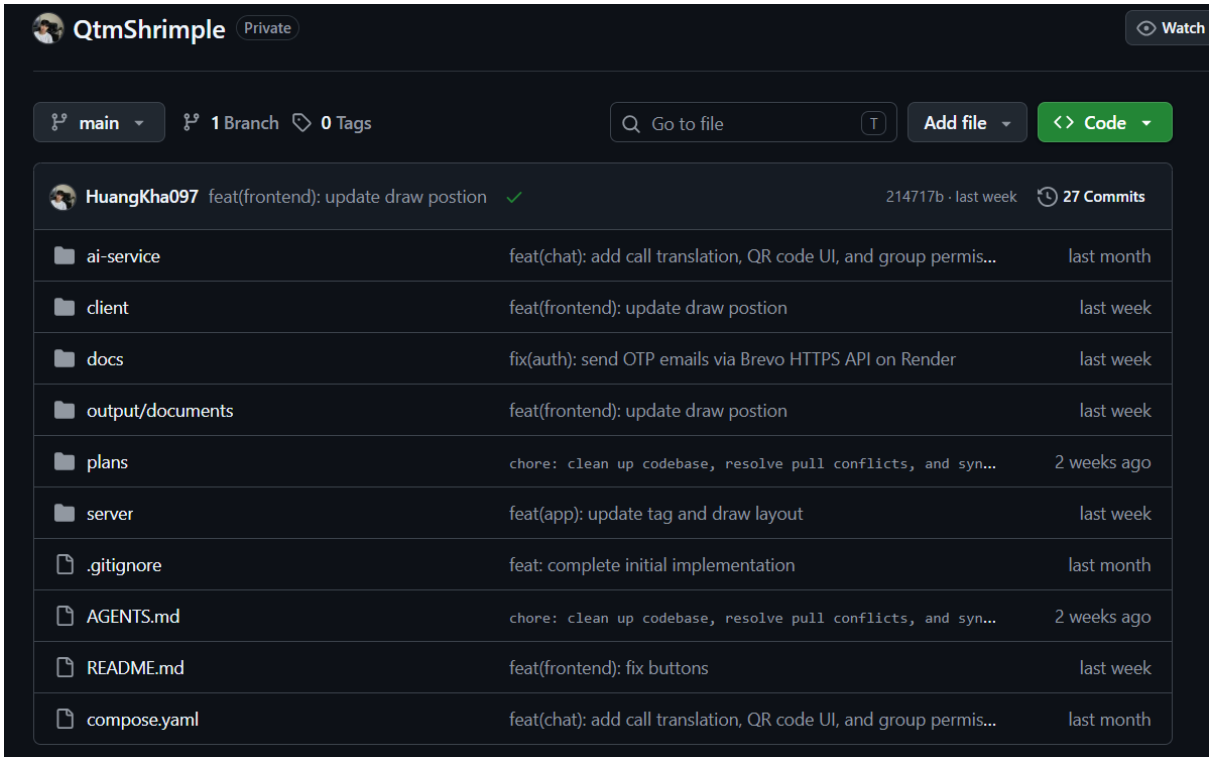
Thành phần	Vị trí hoặc công nghệ	Trách nhiệm
Mã nguồn	GitHub HuangKha097/QtmShrimple	Quản lý repository, nhánh và commit
Frontend	React 19 và Vite 7 trên Vercel	Build và phân phối SPA
Backend	Node.js ESM, Express 5, Socket.IO 4 trên Render	Xác thực, API, quyền và sự kiện thời gian thực

Thành phần	Vị trí hoặc công nghệ	Trách nhiệm
Dữ liệu	MongoDB Atlas, Mongoose 8	Lưu dữ liệu người dùng và nội dung
Email	Brevo qua backend	Gửi email giao dịch có OTP
Media	Cloudinary qua backend	Lưu và cung cấp ảnh, video

Các phiên bản công nghệ nêu theo nhánh phiên bản trong hồ sơ package của dự án, không phải phép kiểm kê trực tiếp toàn bộ runtime trên cloud. Các thành phần camera, cuộc gọi và AI có dependency riêng ngoài phạm vi build frontend. [10]

3.2 Tổ chức mã nguồn và đơn vị triển khai

Repository được tổ chức thành client, server và ai-service. Thư mục client chứa frontend với package và lockfile riêng; server chứa backend; ai-service phục vụ các tính năng AI tùy chọn. Cách tách này dẫn đến hai quy trình triển khai chính độc lập: Vercel build client và Render chạy server. [10]



Hình 3.2. Cấu trúc thư mục dự án

Lockfile phải đồng bộ với package.json để npm ci tái tạo dependency. Tài nguyên tải động trong public cần được giữ nếu ứng dụng còn tham chiếu tới chúng; việc không xuất hiện trong import JavaScript chưa đủ căn cứ để loại một tệp khỏi bản phát hành. [10]

3.3 Ranh giới trách nhiệm giữa các thành phần

Trình duyệt chịu trách nhiệm hiển thị và thu nhận thao tác, nhưng không phải nơi quyết định cuối cùng về quyền truy cập. Backend nhận danh tính, kiểm tra nghiệp vụ, xử lý dữ liệu và gọi dịch vụ ngoài. MongoDB lưu trạng thái, còn Brevo và Cloudinary cung cấp khả năng chuyên biệt. Phân chia trách nhiệm như vậy giúp xác định thành phần phải xác nhận một yêu cầu trước khi tạo tác động. [10]

Ví dụ, việc ẩn nút trên giao diện chỉ điều chỉnh trải nghiệm người dùng; một request gửi trực tiếp vẫn phải được backend kiểm tra. Tương tự, URL media được hiển thị không thay cho quyền tạo hoặc xóa nội dung liên quan. Trong thiết kế nghiệm thu, cần kiểm tra từ một người có quyền và một người không có quyền đối với cùng tài nguyên, thay vì chỉ xác nhận giao diện của tài khoản chủ sở hữu. [10]

3.4 Các nhóm dữ liệu được quan sát

Danh sách collection trên Atlas có thể nhóm theo nghiệp vụ để giải thích quan hệ với các màn hình ứng dụng. Bảng dưới chỉ tổng hợp tên collection đang thấy trong ảnh; các mô tả là chức năng ở cấp nhóm và không thay thế việc kiểm tra schema hoặc document thực tế. [10]

Bảng 3.2. Nhóm collection theo chức năng trong ảnh MongoDB Atlas

Nhóm dữ liệu	Collection trong ảnh	Liên hệ nghiệp vụ
Người dùng và phiên	users, sessions, otps	Danh tính, phiên truy cập và mã xác minh
Quan hệ xã hội	friends, friendrequests	Quan hệ bạn bè và lời mời
Trao đổi	conversations, messages	Hội thoại và nội dung tin nhắn
Nội dung công khai	posts, hashtags	Bài viết và phân loại theo hashtag

Nhóm dữ liệu	Collection trong ảnh	Liên hệ nghiệp vụ
Thông báo	notifications	Thông tin sự kiện gửi đến người dùng

Thiết kế chi tiết cần tiếp tục xác nhận các trường bắt buộc, cách tham chiếu người tham gia, quy tắc xóa và index hỗ trợ truy vấn. Báo cáo không dựng sơ đồ quan hệ trường dữ liệu khi chưa có schema đối chiếu. Danh sách nhóm ở đây giúp dẫn dắt phần kiểm chứng và xác định phạm vi cần bổ sung.

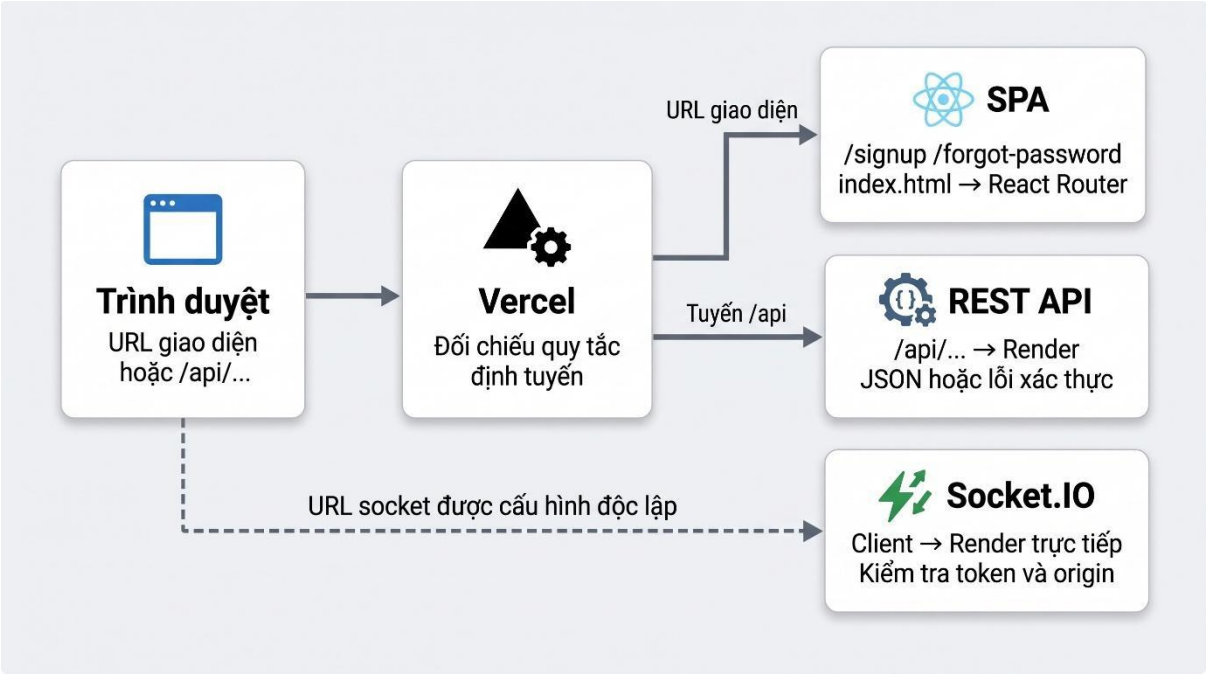
3.5 Thiết kế định tuyến và cấu hình Vercel

Quy tắc /api được đặt trước fallback SPA. Trong tệp client/vercel.json, framework, lệnh cài đặt, lệnh build và thư mục đầu ra được khai báo cùng hai rewrite dưới đây. [10]

```json
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "framework": "vite",
  "installCommand": "npm ci",
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "rewrites": [
    {
      "source": "/api/:path*",
      "destination": "https://shrimple.onrender.com/api/:path*"
    },
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

Hình 3.3. Nội dung tệp cấu hình client/vercel.json

Ví dụ, /api/users/me được chuyển tới cùng route trên Render, trong khi /signup được trả index.html để React Router dựng biểu mẫu. Root Directory vẫn phải chọn client tại thiết lập dự án; tệp cấu hình này không thay cho việc xác định đúng thư mục ứng dụng. [10]



Hình 3.4. Phân luồng URL giao diện REST API và Socket.IO

Hình 3.4 thể hiện cơ chế cùng origin theo cấu hình. Giá trị VITE_API_URL trên cloud có thể thay đổi đích REST mà trình duyệt gọi; ảnh Dashboard đang che giá trị nên chưa xác nhận mọi request thực tế của client đều đi qua rewrite. Socket.IO được cấu hình độc lập và cần kiểm tra URL, origin cùng token riêng. [10]

3.5.1 Đối chiếu các đường kết nối khi đổi môi trường

Khi thay domain frontend hoặc chuyển backend, cần kiểm tra ba loại địa chỉ riêng. REST có thể dùng cùng origin hoặc URL tuyệt đối; Socket.IO có địa chỉ bắt tay riêng; tài nguyên có thể là URL hoàn chỉnh do Cloudinary trả về hoặc đường dẫn cần ghép tiền tố. Một thay đổi cấu hình chỉ đúng khi phù hợp với cách sử dụng địa chỉ tại thành phần tương ứng. [10]

Bảng 3.3. Các đường kết nối cần đối chiếu giữa môi trường

Luồng	Đường đi cần đối chiếu	Điểm kiểm tra
REST cùng origin	Trình duyệt → Vercel /api → Render	Rewrite, JSON, phiên và chính sách cache
REST trực tiếp	Trình duyệt → URL backend	CORS, credentials và hợp đồng endpoint
Socket.IO	Trình duyệt → URL Render riêng	Origin, token handshake và reconnect

Luồng	Đường đi cần đối chiếu	Điểm kiểm tra
Tài nguyên media	Trình duyệt → URL tài nguyên được trả về	URL hợp lệ, loại tệp và quyền nghiệp vụ liên quan

Ở môi trường Preview, việc có URL mới giúp tách bản kiểm tra nhưng đồng thời tạo một origin mới. Backend cần chính sách cho phép phù hợp nếu client kết nối trực tiếp; tên miền Preview không mặc nhiên được chấp nhận. Cần ghi rõ URL Preview trong kết quả để có thể phân biệt lỗi của bản đang thử với bản Production. [1][7][10]

3.6 Phân tách cấu hình frontend và backend

Bảng 3.4. Phạm vi sử dụng các biến cấu hình chính

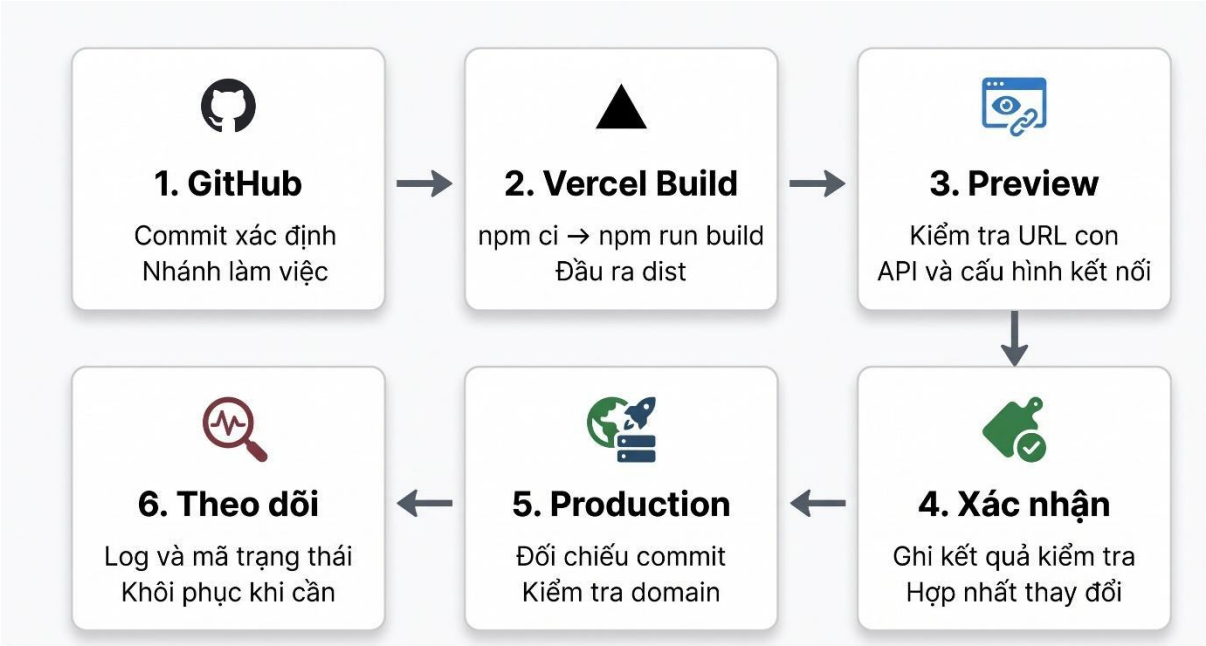
Biến hoặc nhóm biến	Nơi sử dụng	Vai trò
VITE_API_URL	Frontend	Base URL của Axios; có cơ chế mặc định cùng origin theo source
VITE_SOCKET_URL	Frontend	Đích kết nối Socket.IO
VITE_ASSET_BASE_URL	Frontend	Tiền tố tài nguyên tương đối
VITE_GOOGLE_CLIENT_ID	Frontend	Định danh OAuth công khai
CLIENT_ORIGINS	Backend	Origin được Express và Socket.IO cho phép
MONGODB_ CONNECTIONSTRING	Backend	Chuỗi kết nối cơ sở dữ liệu
ACCESS_TOKEN_SECRET và các khóa dịch vụ	Backend	Ký token, xác thực Brevo và Cloudinary

Trong source, Axios bật withCredentials; socket có quá trình bắt tay và kiểm tra token. Cookie refresh sử dụng httpOnly và có thiết lập production riêng. Hành vi cookie còn phụ thuộc trình duyệt và đường đi request, nên cần kiểm tra đăng nhập, làm mới phiên và đăng xuất thành một luồng hoàn chỉnh. [7][10]

3.7 Quy trình quản lý cập nhật

Một bản cập nhật cần có commit xác định, kết quả kiểm tra tương ứng và deployment để đối chiếu. Quy trình ở Hình 3.5 được đề xuất cho các lần cập nhật tiếp

theo, giúp liên hệ thay đổi mã nguồn với bản Preview, bản Production và kết quả sau phát hành. [1]



Hình 3.5. Quy trình đề xuất để quản lý cập nhật frontend

Sau khi phát hành, hồ sơ kiểm tra cần ghi domain, URL deployment, nhánh, commit, thời điểm và chức năng chịu ảnh hưởng. Nếu cần khôi phục, chọn deployment ổn định đủ điều kiện và đánh giá khả năng tương thích backend trước khi áp dụng. [1]

3.8 Luồng xác thực email và các điểm đối chiếu

Luồng OTP bắt đầu từ thao tác đăng ký hoặc khôi phục mật khẩu trên frontend. Backend xử lý yêu cầu và thông tin xác minh, đồng thời sử dụng dịch vụ email để chuyển mã đến người nhận. Người dùng nhập lại mã; backend kiểm tra điều kiện hợp lệ trước khi chuyển sang bước nghiệp vụ tiếp theo. Sơ đồ sau tập trung vào các điểm cần đối chiếu giữa giao diện, xử lý và dịch vụ gửi thư. [5][10]

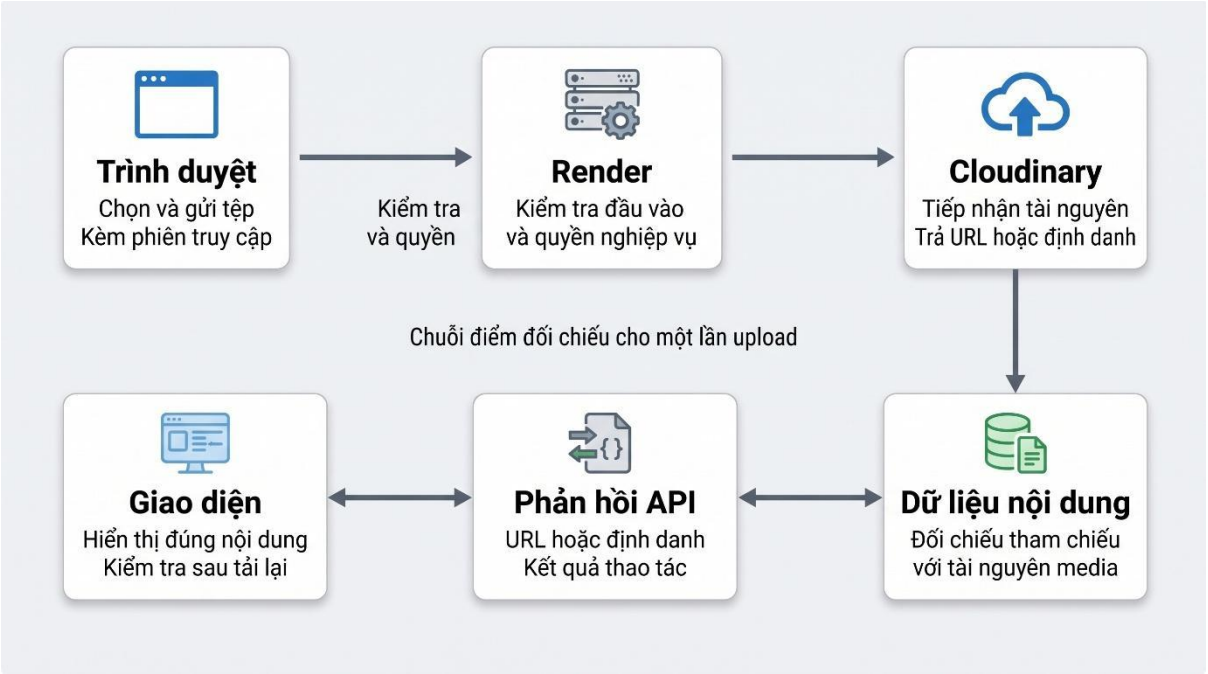


Hình 3.6. Luồng OTP và các điểm đối chiếu kết quả gửi thư

Bằng chứng của luồng phải liên hệ cùng một lần yêu cầu. Tối thiểu cần ghi nhận kết quả backend, trạng thái gửi phù hợp ở Brevo và thư nhận tại hộp thư thử. Khi thử mã sai hoặc hết hạn, kết quả mong đợi là từ chối chuyển trạng thái nghiệp vụ; không chỉ hiển thị một thông báo ở client. Cần che mã và địa chỉ người nhận khi dùng ảnh làm minh chứng.

3.9 Luồng media từ upload đến hiển thị

Trong thiết kế hiện có, backend tiếp nhận tệp qua Multer và có helper upload Cloudinary. Luồng kiểm tra cần nối thao tác chọn tệp, kết quả upload, thông tin tài nguyên và nội dung xuất hiện trên giao diện. Việc tách điểm xử lý tệp khỏi frontend cho phép kiểm tra quyền và điều kiện đầu vào trước khi tạo hoặc gắn tài nguyên với nội dung. [10]



Hình 3.7. Luồng đối chiếu upload và hiển thị tài nguyên media

Một trường hợp upload đạt yêu cầu cần trả URL hoặc định danh đối chiếu được, hiển thị đúng nội dung và duy trì liên kết sau khi tải lại. Trường hợp lỗi cần làm rõ bước nào thất bại: nhận tệp, gọi dịch vụ, ghi dữ liệu hay tải URL. Cách đối chiếu này còn hỗ trợ nhận diện tài nguyên đã tạo nhưng chưa gắn được vào nội dung, để có phương án dọn dẹp phù hợp.

CHƯƠNG 4: TRIỂN KHAI VÀ KẾT QUẢ QUAN SÁT

4.1 Thiết lập frontend và liên kết GitHub

Dự án Vercel shrimple-chat liên kết với repository HuangKha097/QtmShrimple. Root Directory được đặt là client; framework là Vite; đầu ra là dist. Các thiết lập này phù hợp với đơn vị frontend đã thiết kế ở Chương 3. [10]

Bảng 4.1. Cấu hình triển khai frontend trên Vercel

Thiết lập	Giá trị đối chiếu	Ý nghĩa
Repository	HuangKha097/ QtmShrimple	Nguồn mã của ứng dụng
Root Directory	client	Phạm vi chạy lệnh frontend
Framework Preset	Vite	Công cụ build giao diện
Install Command	npm ci	Cài dependency theo lockfile
Build Command	npm run build	Gọi script build của client
Output Directory	dist	Thư mục tài nguyên được phát hành
Production Branch	main	Nhánh hiển thị trong bộ quan sát

4.1.1 Trình tự triển khai và điều kiện chuyển bước

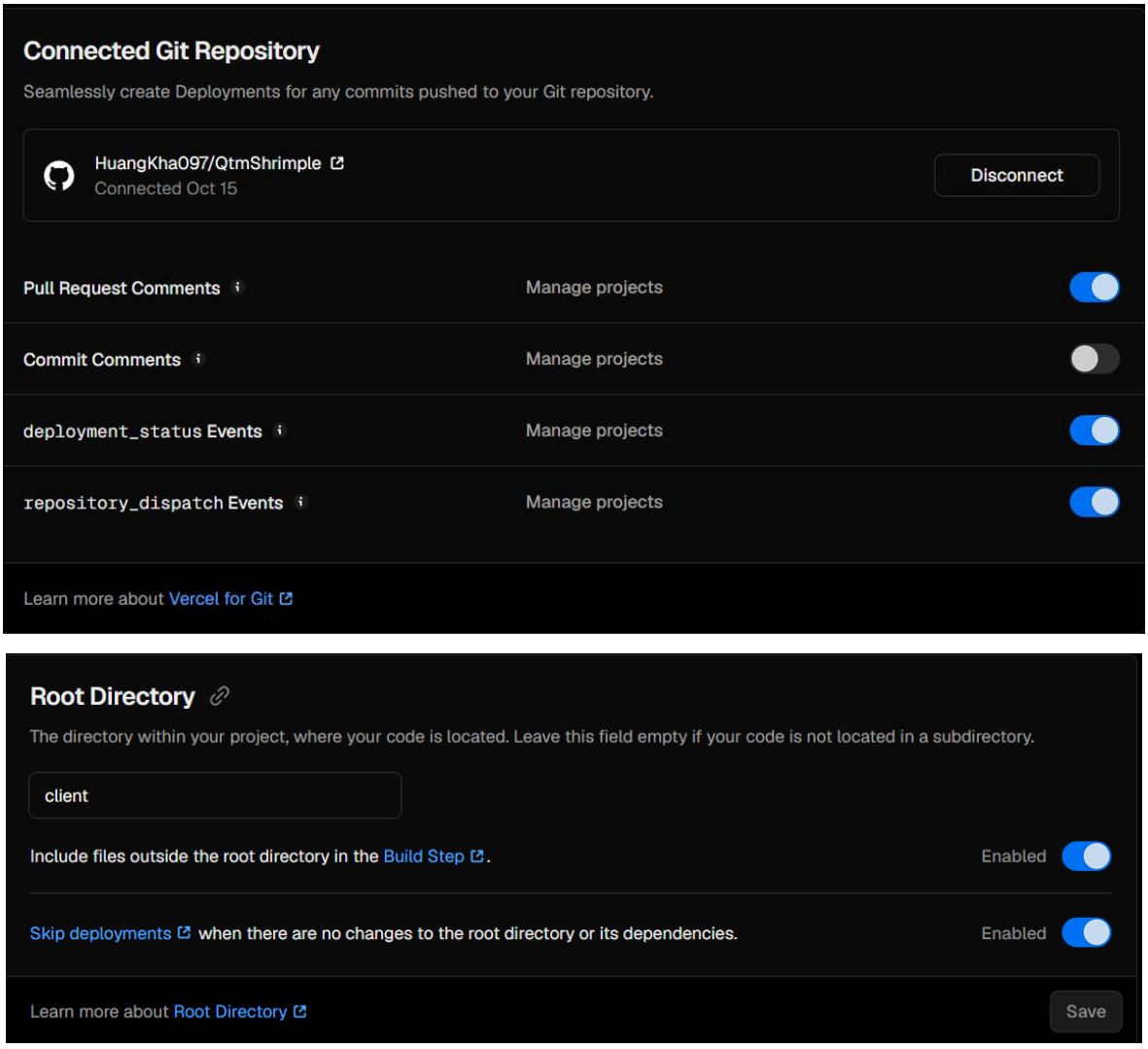
Bước chuẩn bị bắt đầu từ kiểm tra thư mục frontend và dependency. client cần có package.json, lockfile, index.html cùng src và public theo cấu trúc dự án. Lệnh build phải tạo đúng đầu ra trước khi đánh giá cấu hình phát hành. Nếu quy trình cục bộ không tạo được dist, cần xử lý nguyên nhân ở package hoặc mã nguồn trước khi suy lỗi do dịch vụ cloud. [10]

Khi liên kết repository, phạm vi cấp quyền cần đủ để Vercel đọc đúng kho mã; Root Directory sau đó giới hạn ứng dụng được build. Với repository nhiều thành phần, chọn nhầm thư mục có thể làm nền tảng đọc package không tương ứng hoặc không tìm thấy output. Điều kiện chuyển sang bước cấu hình là xác nhận dự án đang hướng tới package client và nhánh dùng cho Production. [1][10]

Ở bước cấu hình, đối chiếu Framework Preset, Install Command, Build Command và Output Directory với tệp dự án. Nếu Dashboard cảnh báo Production Overrides, cần xác định cấu hình nào thuộc bản đang chạy và cấu hình nào dành cho lần triển khai kế

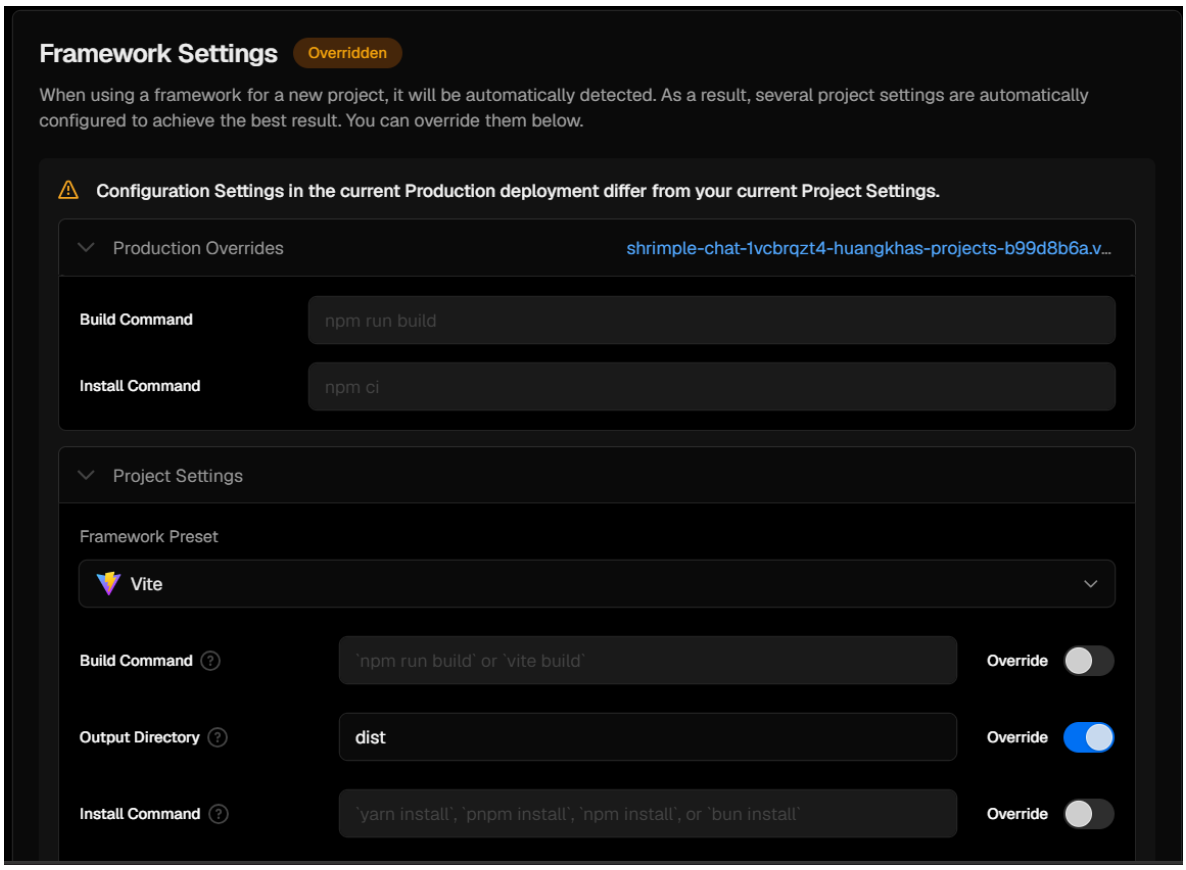
tiếp. Không nên sửa một giá trị rồi dùng trạng thái Ready của bản trước làm bằng chứng thay đổi đã có hiệu lực. [1]

Sau khi khai báo biến môi trường và tạo deployment, kiểm tra lần lượt log, địa chỉ của bản cụ thể, domain Production và các request ứng dụng. Mỗi bước trả lời một câu hỏi khác nhau: build có tạo sản phẩm không, domain đang phục vụ bản nào, HTML và tài nguyên có tải đúng không, API cùng các tích hợp có phản hồi theo điều kiện thử không. Kết quả được lưu cùng commit để hỗ trợ tái lập.



Hình 4.*1. Repository liên kết và thư mục gốc của frontend*

Ảnh framework đồng thời hiển thị Production Overrides và cảnh báo cấu hình của deployment Production khác Project Settings hiện tại. Do đó, khi phân tích một bản đang chạy, phải đọc cấu hình gắn với bản đó và log thực thi, thay vì chỉ dùng các giá trị mặc định của dự án.



Hình 4.2. Framework Vite và cấu hình riêng của bản Production

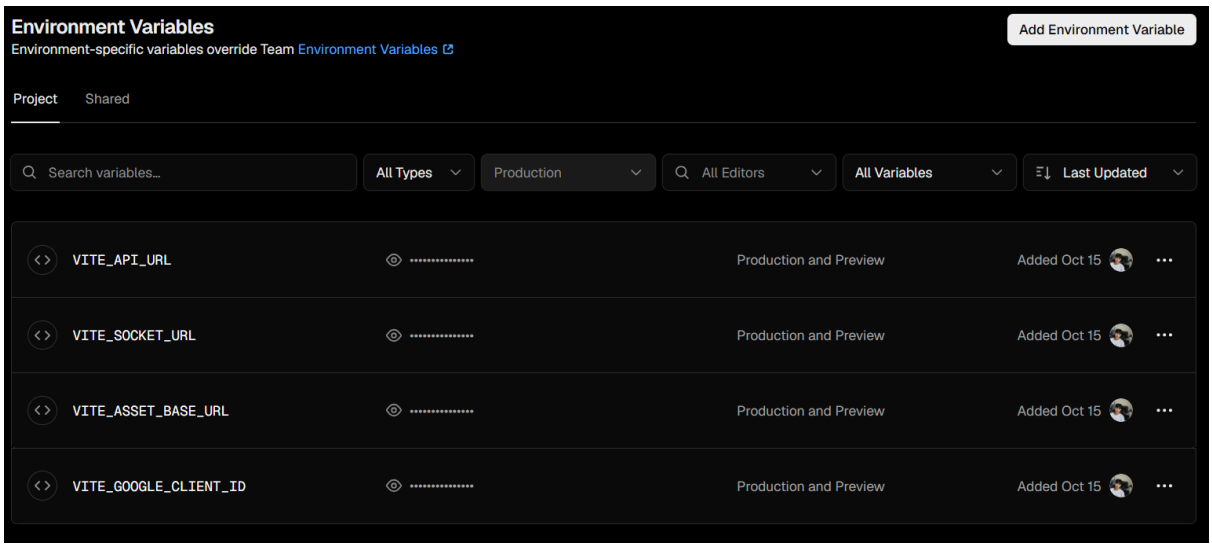
4.2 Chuẩn bị bản build và cấu hình môi trường

Các lệnh ở Hình 4.3 mô tả quy trình chuẩn bị frontend từ thư mục gốc repository theo script dự án. Lệnh test hiện có tập trung vào module tracker; kết quả của nó không đại diện toàn bộ nghiệp vụ hoặc thiết bị camera. Các luồng backend cần có kiểm tra riêng. [10]

```
user@developer-pc:~/projects/web-app$ npm --prefix client ci
user@developer-pc:~/projects/web-app$ npm --prefix client run lint
user@developer-pc:~/projects/web-app$ npm --prefix client test
user@developer-pc:~/projects/web-app$ npm --prefix client run build
```

Hình 4.3. Các lệnh kiểm tra và chuẩn bị bản build frontend

Dashboard Vercel ghi nhận bốn biến VITE_API_URL, VITE_SOCKET_URL, VITE_ASSET_BASE_URL và VITE_GOOGLE_CLIENT_ID với phạm vi Production and Preview. Ảnh xác nhận tên và phạm vi của các biến, còn giá trị đang được che. Sau khi thay đổi biến frontend, cần build deployment mới và thử lại kết nối tương ứng. [1][2]



Hình 4.4. Các biến môi trường frontend trong Production và Preview

4.2.1 Kiểm tra hiệu lực của cấu hình môi trường

Tên biến và phạm vi trong Dashboard là điểm bắt đầu của việc đối chiếu, còn đích request thực tế là bằng chứng cấu hình đang được sử dụng. Với frontend, cần xem deployment nào đã build sau thay đổi; với backend, cần xác định tiến trình đã sử dụng cấu hình tương ứng. Khi có nhiều môi trường, hồ sơ kiểm tra phải chỉ rõ đang xem Production hay Preview. [1][2]

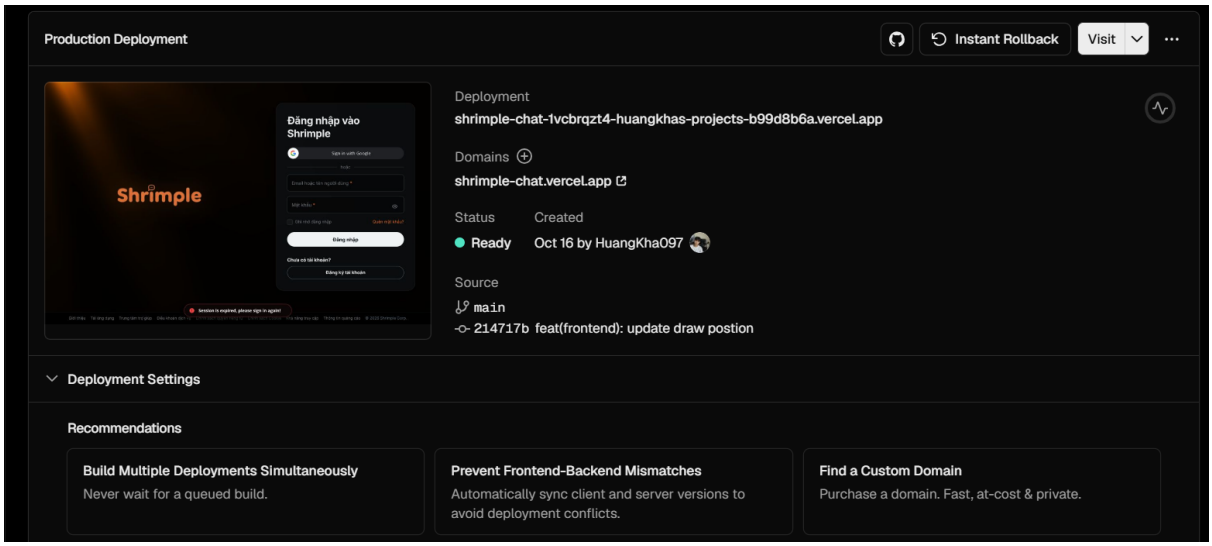
Đối với REST, kiểm tra URL đầy đủ của request và đường dẫn /api. Đối với Socket.IO, kiểm tra địa chỉ bắt tay và lỗi kết nối riêng. Đối với media, kiểm tra URL do backend trả và khả năng tải tài nguyên. Nếu chỉ nhìn thấy một chức năng hoạt động, chưa đủ kết luận cả ba nhóm địa chỉ đã đồng bộ. Đây là lý do bảng biến cấu hình được tổ chức theo nơi sử dụng thay vì chỉ liệt kê tên. [10]

Quản lý bí mật cần đi cùng vòng đời tích hợp. Khóa được cấp cho backend phải giữ ở phía backend; khi thay khóa, cần kiểm tra dịch vụ gọi đi và thu hồi khóa cũ theo kế hoạch phù hợp. Ảnh chỉ cần thể hiện tên biến và trạng thái; việc che giá trị trên Dashboard không bảo vệ một bí mật đã bị đưa vào bundle frontend. [2][10]

4.3 Bản Production và khả năng truy vết phiên bản

Ảnh Production trong đợt khảo sát bổ sung ngày 16/10/2026 hiển thị Ready, nhánh main và commit 214717b với mô tả “feat(frontend): update draw postion”. Domain được gán là shrimple-chat.vercel.app. Đây là mốc phiên bản của ảnh này; các quan sát trước

đó đã ghi nhận commit eb57fe4. Hai mã thuộc các thời điểm khác nhau và không được ghép thành một định danh.

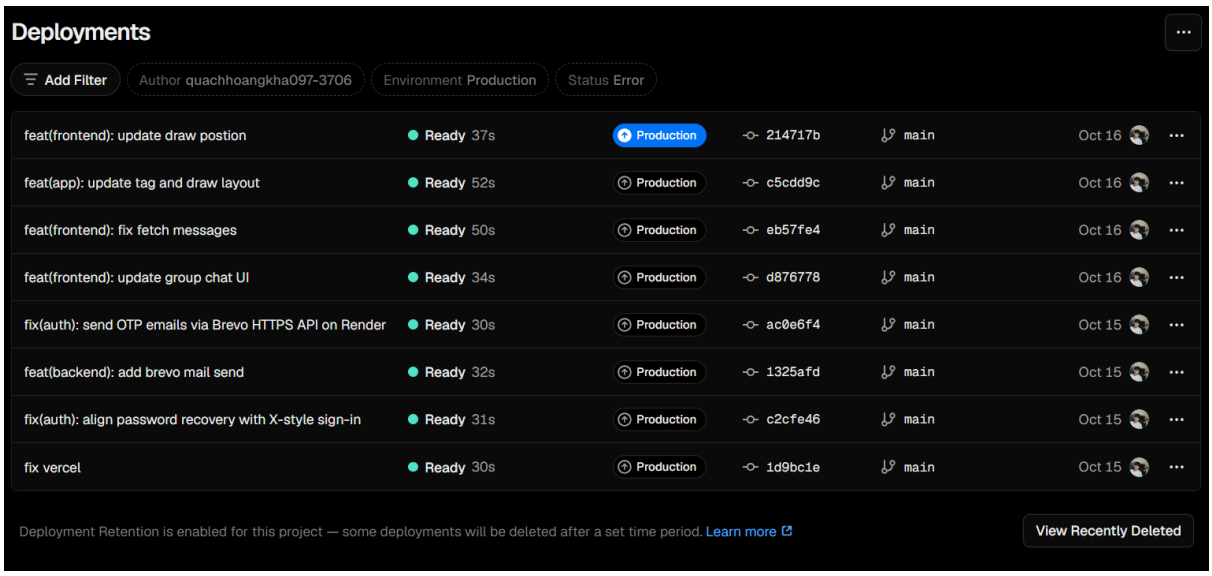


Hình 4.5. Bản Production Vercel tại commit 214717b

Domain Production trỏ tới bản được gán tại thời điểm truy cập, trong khi URL deployment nhận diện một bản cụ thể. Lịch sử triển khai và commit là cơ sở truy vết khi kết quả kiểm tra thay đổi sau cập nhật. Trạng thái Ready cho biết bản triển khai hoàn tất theo nền tảng; kết quả nghiệp vụ cần đối chiếu thêm ở các lớp phía sau. [1]

4.3.1 Lịch sử phát hành và quan hệ giữa các mốc phiên bản

Danh sách Deployments ghi các bản Ready trên main cùng commit và thời gian. Bộ ảnh có thứ tự 214717b, c5cdd9c và eb57fe4 ở các mốc quan sát tương ứng. Điều này giải thích vì sao một ảnh Overview sớm hơn có thể khác ảnh chi tiết được thu thập sau đó mà không tạo mâu thuẫn về định danh.



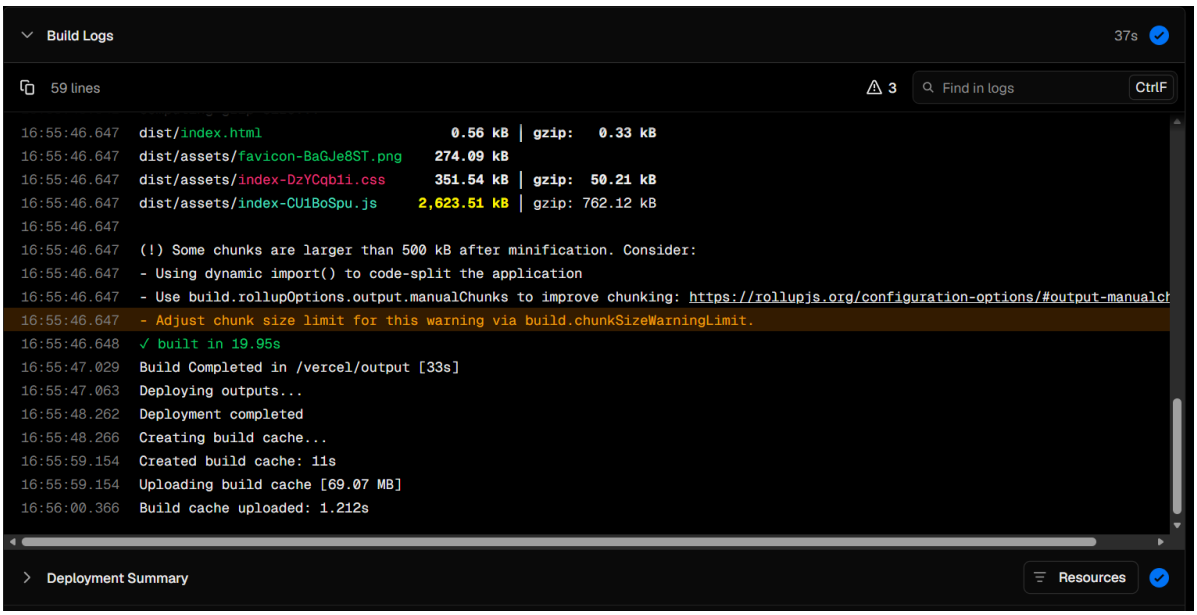
Hình 4.6. Lịch sử deployment Vercel và các mốc commit

Khi liên hệ thay đổi với sự cố, cần dùng định danh của cả frontend và backend. Một bản giao diện mới có thể chạy cùng backend chưa cập nhật; ngược lại, backend có thể đổi hợp đồng API trước giao diện. Vì vậy, hồ sơ phát hành nên ghi cặp phiên bản cùng kết quả kiểm tra các chức năng chịu tác động, thay vì yêu cầu hai dịch vụ luôn có cùng commit.

4.4 Phân tích log build và tài nguyên đầu ra

Cấu hình bản 214717b sử dụng npm ci cho bước cài dependency; log thể hiện tiến trình chuyển sang vite build. Đoạn log được lưu còn có gợi ý chạy npm audit, cảnh báo install-scripts và Unknown env config cache-dir. Khi rà soát, cần phân biệt thông báo cài đặt, kết quả kiểm tra dependency và lỗi làm dừng tiến trình; không dùng một dòng warning để kết luận build thất bại. [10]

Giai đoạn build ghi Vite v7.3.0 và 3506 module được biến đổi. Phần cuối log liệt kê index.html, CSS và JavaScript trong dist, đồng thời cho biết deployment hoàn tất.



Hình 4.7. Log tạo bundle và hoàn tất deployment Vercel

Bảng 4.2. Các chỉ tiêu build được ghi nhận

Chỉ tiêu trong log	Giá trị	Cách diễn giải
JavaScript chính	2623,51 kB; gzip 762,12 kB	Kích thước bundle của bản build
CSS	351,54 kB; gzip 50,21 kB	Kích thước stylesheet đầu ra
Vite build	19,95 giây	Thời gian Vite báo cho bước build
Build Completed	Khoảng 33 giây	Mốc hoàn thành build trong log
Deployment	37 giây	Thời gian ở giao diện chi tiết bản triển khai

Cảnh báo chunk lớn hơn 500 kB gợi ý xem xét tách mã và tải thư viện theo nhu cầu. Tuy nhiên, kích thước bundle và thời gian build không phải thời gian phản hồi HTTP hay thời gian hiển thị trang trên thiết bị người dùng. Cần phép đo trải nghiệm riêng trước khi đánh giá hiệu năng.

4.4.1 Phân tích cảnh báo và ưu tiên xử lý

Trong log, warning về cấu hình môi trường npm, kết quả audit và cảnh báo chunk lớn thuộc các nhóm vấn đề khác nhau. Dòng warning không tự quyết định build thất bại; cần xem tiến trình có tiếp tục, mã thoát và trạng thái cuối. Trong bản được chụp, tiến trình tiếp tục tạo bundle và hoàn tất deployment.

npm audit hỗ trợ nhận diện advisory trong cây dependency. Khi có kết quả audit, nhóm cần xác định package, phiên bản, quan hệ phụ thuộc và ảnh hưởng của phương án nâng cấp. Không nên dùng một con số tổng làm kết luận hệ thống đã an toàn hoặc đã bị khai thác; cũng cần kiểm tra lại build và chức năng chịu tác động sau khi thay dependency. [8]

4.4.2 Đối chiếu tài nguyên được phát hành

Trang Resources trong bộ ảnh ghi nhận 18 Static Assets, gồm index.html, bundle JavaScript, CSS, logo và các tệp công khai. Đây là lớp kiểm tra sau build: một tệp xuất hiện trong log cần được đối chiếu với output phục vụ khi điều tra lỗi tải tài nguyên. Số lượng trong ảnh chỉ mô tả danh sách được ghi nhận của deployment.

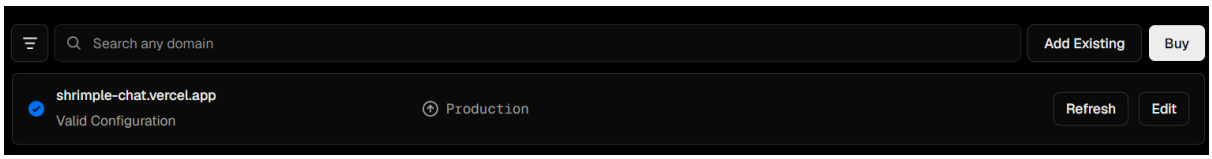


Hình 4.8. Danh sách Static Assets của deployment frontend

Nếu HTML tải được nhưng màn hình trắng, cần đọc lỗi JavaScript và các request bundle. Nếu chỉ thiếu một ảnh hoặc mô hình tải động, cần kiểm tra đường dẫn và tài nguyên public tương ứng. Việc phân phối thành công HTML không loại trừ lỗi tham chiếu tệp; ngược lại, một lỗi tài nguyên đơn lẻ cũng không đồng nghĩa toàn bộ deployment chưa được phát hành. [10]

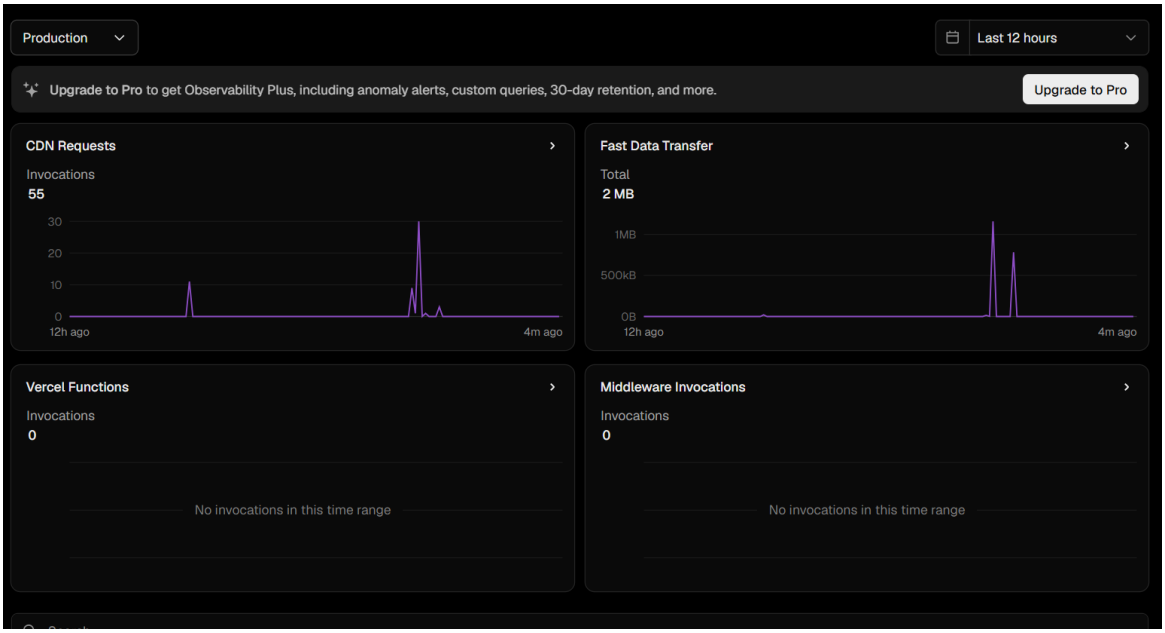
4.5 Tên miền và dữ liệu quan sát vận hành

Domain shrimple-chat.vercel.app được gán Production và hiển thị Valid Configuration. Đây là bằng chứng về tên miền trong môi trường khảo sát. Đề tài sử dụng subdomain do Vercel cấp; chưa có minh chứng cấu hình tên miền riêng.



Hình 4.9. Tên miền Production có cấu hình hợp lệ

Observability trong khung Last 12 hours ghi 55 lượt ở ô CDN Requests và tổng Fast Data Transfer là 2 MB. Hai ô Vercel Functions và Middleware Invocations đều ghi 0, kèm thông báo không có lượt gọi trong khoảng thời gian này. Các giá trị phản ánh nhóm dữ liệu và bộ lọc đang chọn; backend của mô hình được vận hành riêng trên Render. [1]



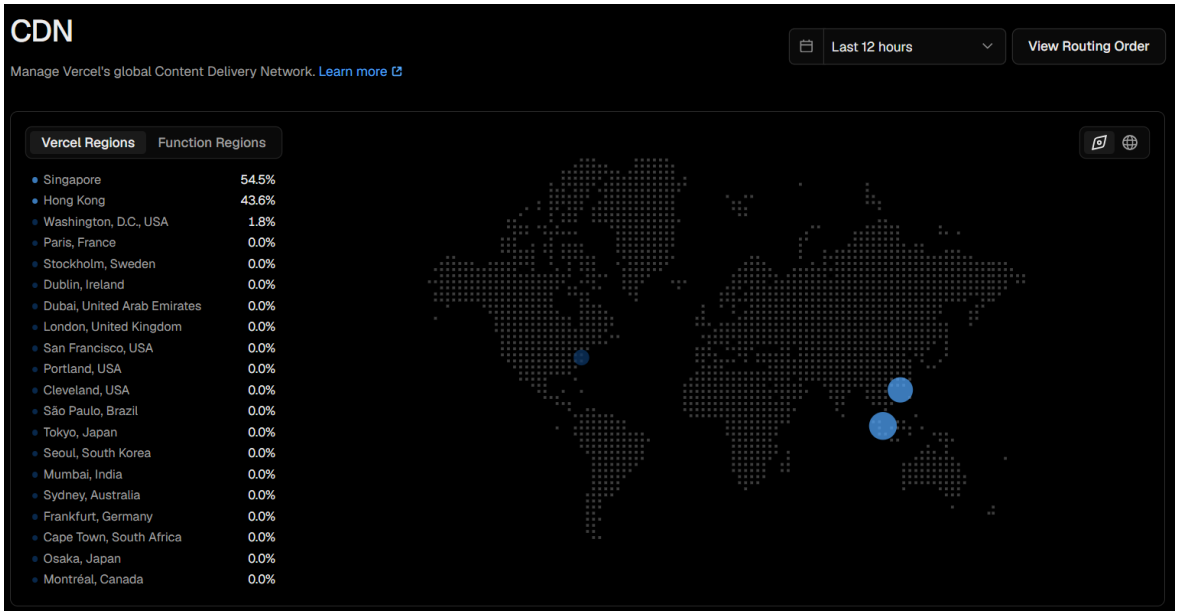
Hình 4.10. Dữ liệu Observability theo môi trường và khoảng thời gian

Màn hình Web Analytics ở trạng thái Enable Web Analytics và có DEMO DATA; Speed Insights còn hiển thị hướng dẫn tích hợp React. Vì vậy, báo cáo không dùng các số liệu mẫu làm lượng truy cập thực hoặc đưa ra kết luận Core Web Vitals. Khi triển khai đo lường, cần xác định thời gian thu thập và giữ cùng điều kiện khi so sánh. [1]

Các khu vực trên bản đồ CDN mô tả nơi phục vụ lưu lượng trong khoảng thời gian chọn, còn mã vùng iad1 trong build log là nơi chạy build. Hai thông tin thuộc các công đoạn khác nhau. Tương tự, “No Routing Rules” ở cấp dự án không phủ định rewrite được khai báo trong cấu hình deployment. [1]

4.5.1 Phân bố CDN và giới hạn diễn giải số liệu

Ảnh CDN trong khung Last 12 hours ghi Singapore 54,5%, Hong Kong 43,6% và Washington D.C. 1,8%; các địa điểm khác đang thấy trong ảnh ghi 0,0%. Các tỷ lệ thuộc khung thời gian đang chọn và phản ánh phân bố được Dashboard hiển thị. Chúng không phải phép đo vị trí địa lý của mọi người dùng hoặc cam kết độ trễ cho các vùng tương ứng.

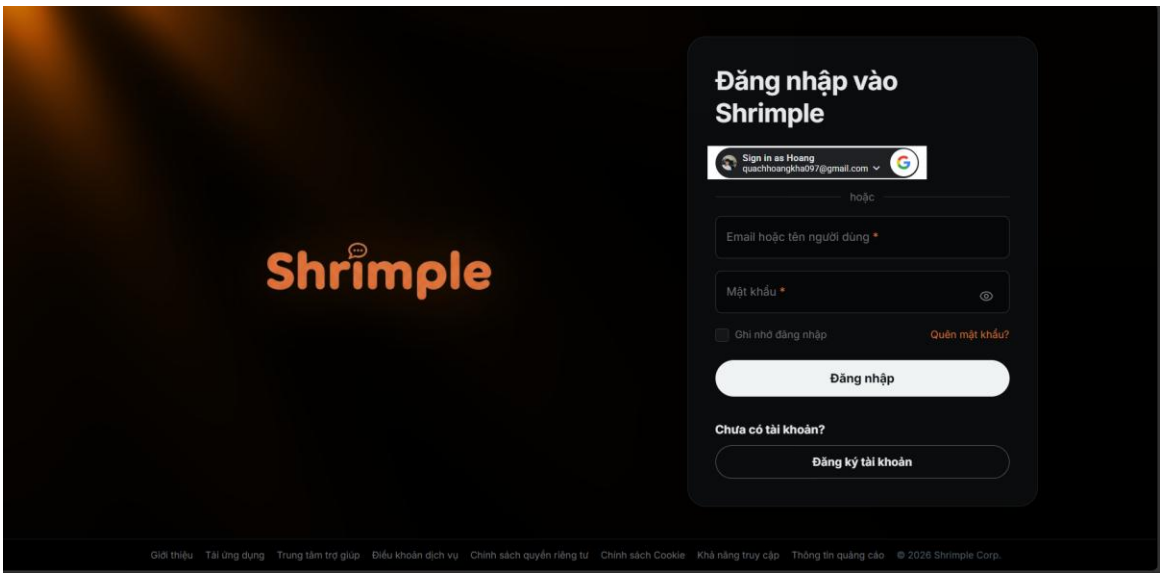


Hình 4.11. Phân bố phục vụ CDN trong khung Last 12 hours

Khi phân tích một đợt truy cập chậm, nên đặt dữ liệu CDN bên cạnh loại request, tài nguyên tải về và trạng thái backend trong cùng khoảng thời gian. Một lần tải frontend nhanh vẫn có thể chờ backend khởi động; một phản hồi API nhanh chưa bảo đảm bundle lớn được tải nhanh trên mạng yếu. Sự phối hợp giữa các lớp là cơ sở chọn phép đo tiếp theo.

4.6 Giao diện ứng dụng và kết quả truy cập HTTP

Khi chưa có phiên đăng nhập, địa chỉ gốc đưa tới màn hình /signin. Bộ quan sát ghi nhận các trang /signup và /forgot-password tiếp tục hiển thị khi truy cập trực tiếp hoặc tải lại. Điều này phù hợp với cơ chế fallback SPA đã thiết kế.



Hình 4.12. Giao diện đăng nhập của Shrimple trên Vercel

Các phép GET được ghi nhận ngày 16/10/2026 lúc 16:14:07 giờ Việt Nam, không gửi token hoặc cookie xác thực. Bảng dưới tổng hợp mã HTTP và kiểu nội dung của từng đường dẫn tại thời điểm này.

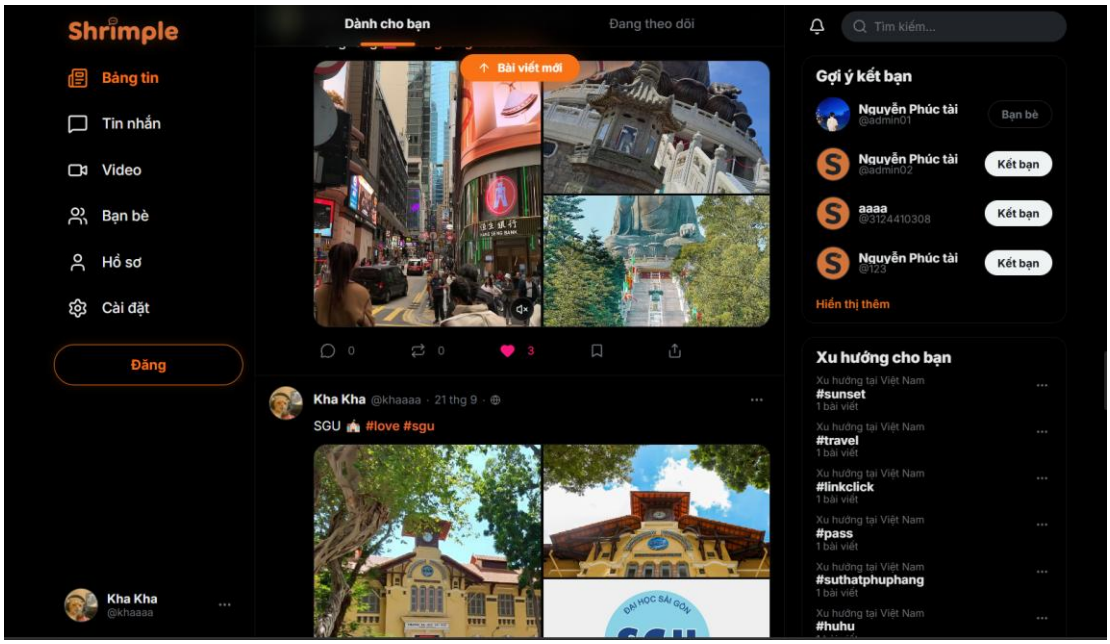
Bảng 4.3. Kết quả HTTP không có phiên đăng nhập ngày 16 tháng 10 năm 2026

Đích và đường dẫn	Mã	Kiểu nội dung	Nhận định
Vercel /	200	text/html	Nhận điểm vào frontend
Vercel /signin	200	text/html	URL con có HTML
Vercel /signup	200	text/html	URL con có HTML
Vercel /forgot-password	200	text/html	URL con có HTML
Vercel /api/users/me	401	application/json	Endpoint yêu cầu xác thực
Render /api/users/me	401	application/json	Backend phản hồi yêu cầu chưa xác thực

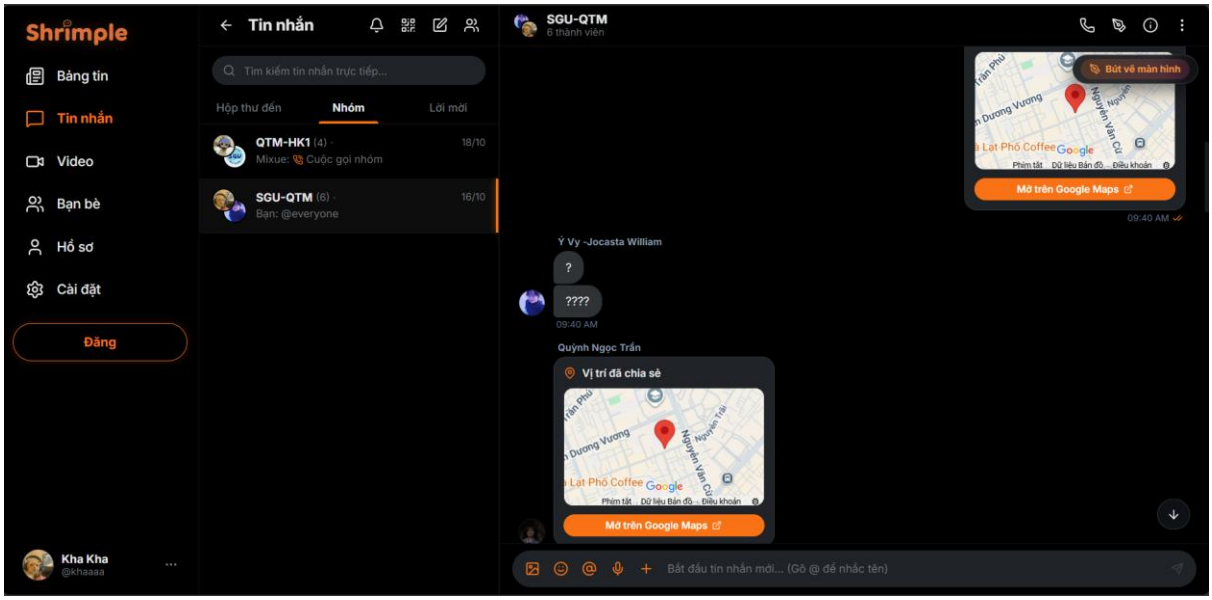
Trong bảng, Vercel là https://shrimple-chat.vercel.app và Render là https://shrimple.onrender.com. Cặp phản hồi 401 JSON cho thấy đường /api được thử không bị trả nhầm HTML của SPA và backend có phản hồi tại thời điểm đó. Mã 401 phù hợp với tình huống chưa xác thực; nó chưa chứng minh đăng nhập hoặc truy cập dữ liệu thành công.

Các ảnh bổ sung thể hiện bảng tin có bài viết, tài nguyên media, tương tác và gợi ý kết bạn; màn hình trò chuyện nhóm hiển thị tin nhắn và thẻ vị trí chia sẻ. Đây là minh

chứng giao diện bên trong ứng dụng, mở rộng phạm vi hình ảnh so với các trang công khai của đợt trước.



Hình 4.13. Bảng tin Shrimple với bài viết và tài nguyên media

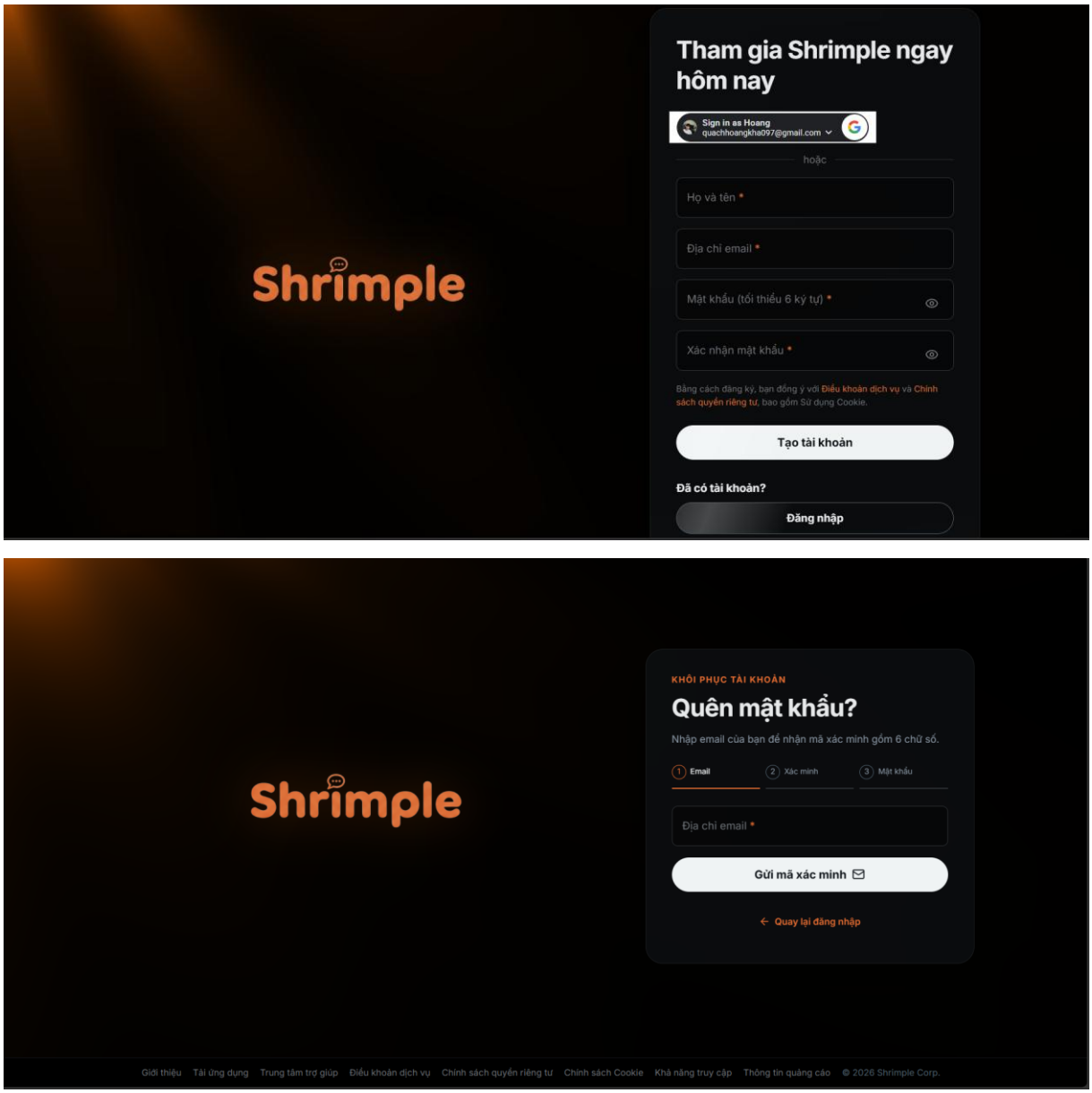


Hình 4.14. Trò chuyện nhóm và nội dung chia sẻ vị trí

Ảnh tĩnh xác nhận trạng thái hiển thị tại thời điểm chụp. Để nghiệm thu gửi nhận realtime, cần một kịch bản có hai tài khoản, thời điểm gửi và nhận, kết quả sau tải lại cùng trường hợp mất kết nối. Hình trò chuyện hiện có chưa chứa đầy đủ chuỗi đối chiếu đó.

4.6.1 Đăng ký và khôi phục mật khẩu trên các URL con

Trang /signup hiển thị biểu mẫu tạo tài khoản, còn /forgot-password hiển thị bước nhập email trong quy trình khôi phục. Hai trang là ví dụ phù hợp để khảo sát fallback vì người dùng có thể đi từ liên kết trong ứng dụng hoặc nhập trực tiếp địa chỉ. Bộ quan sát ngày 16/10/2026 ghi nhận cả hai tiếp tục hiển thị sau tải lại.



Hình 4.15. Giao diện đăng ký và khôi phục mật khẩu tại URL con

Ở mức giao diện, kết quả cho thấy điểm vào React và tài nguyên cần thiết đã tải để dựng biểu mẫu. Ở mức nghiệp vụ, hai quy trình còn cần thao tác gửi biểu mẫu, xác minh email hoặc cập nhật mật khẩu cùng kiểm tra lỗi. Do đó, phép thử tải lại URL được ghi là kết quả routing; luồng OTP được giữ trong kế hoạch kiểm thử đầu cuối.

4.6.2 Cách tái hiện kiểm tra HTTP và đọc phản hồi

Một phép thử HTTP tối thiểu cần lưu mã trạng thái và kiểu nội dung. Các lệnh cURL dưới đây minh họa cách lấy cả header và body cho URL giao diện, API qua domain frontend và API trực tiếp. Khi chạy lại, cần ghi thời điểm cùng trạng thái xác thực; đoạn lệnh chỉ là quy trình tham khảo, không phải kết quả mới được đo. [10]

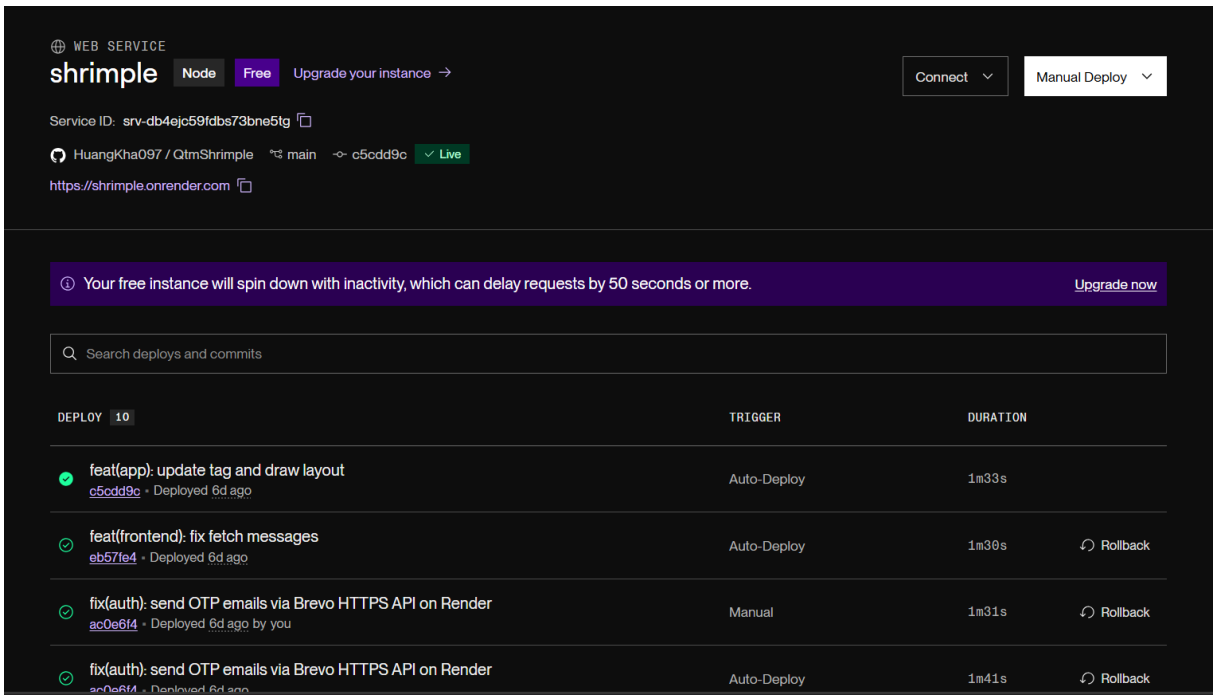


Hình 4.16. Lệnh cURL

Nếu một URL giao diện trả 200 HTML nhưng trình duyệt vẫn trắng, bước tiếp theo là kiểm tra tài nguyên và lỗi JavaScript. Nếu API trả HTML, xem đường dẫn hoặc rewrite; nếu trả JSON cùng 401, đối chiếu điều kiện phiên. Với 5xx hoặc không kết nối được, chuyển sang log backend và trạng thái dịch vụ. Cùng một mã HTTP ở những endpoint khác nhau cần được giải thích theo hợp đồng của endpoint đó.

4.7 Backend Render và cấu hình dịch vụ

Ảnh bổ sung của web service shrimple hiển thị runtime Node, gói Free, nhánh main,commit c5cdd9c và trạng thái Live. URL dịch vụ là https://shrimple.onrender.com. Lịch sử hiển thị các lần Auto-Deploy và deployment liên quan sửa luồng gửi OTP qua Brevo HTTPS API.



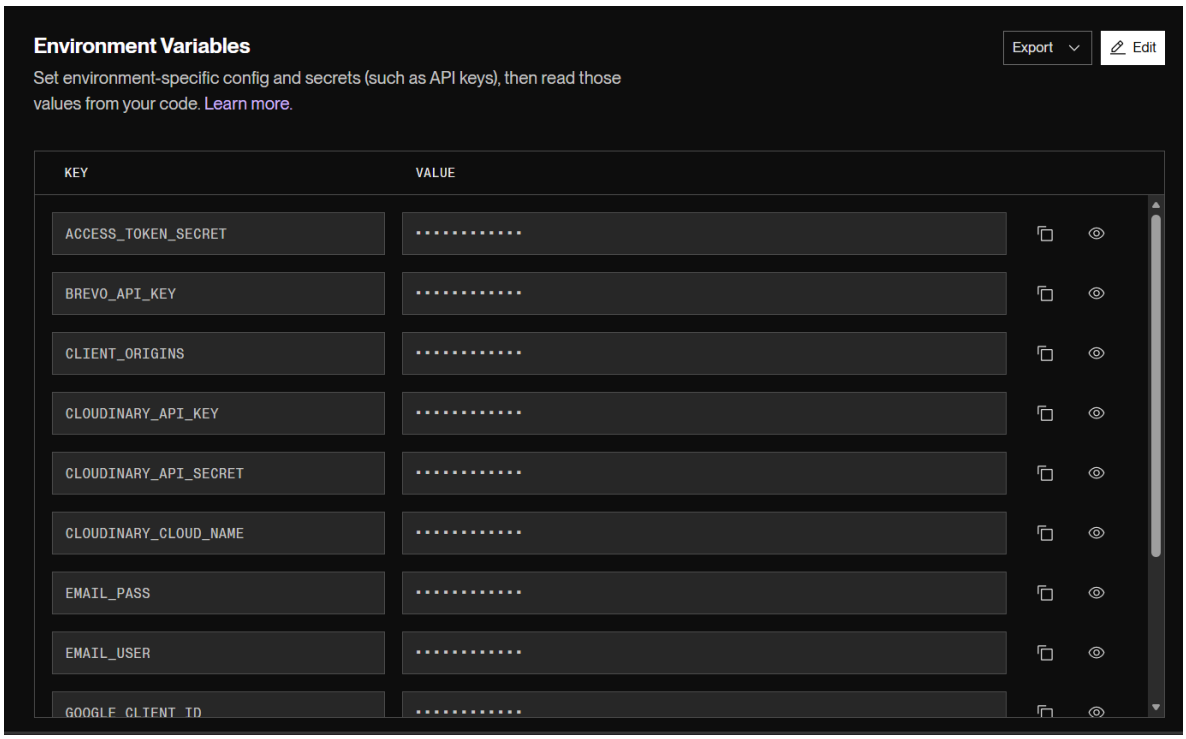
Hình 4.17. Web service Render và lịch sử triển khai backend

Cấu hình Render trong bộ ảnh trước ghi Root Directory là server/. Build backend cài package production và chuẩn bị môi trường Python cho các xử lý có phụ thuộc tương ứng; lệnh khởi động bổ sung môi trường này vào PATH rồi chạy npm start. Các lệnh dưới được chạy trong phạm vi server trên Render, khác với quy trình build client của Vercel. [10]



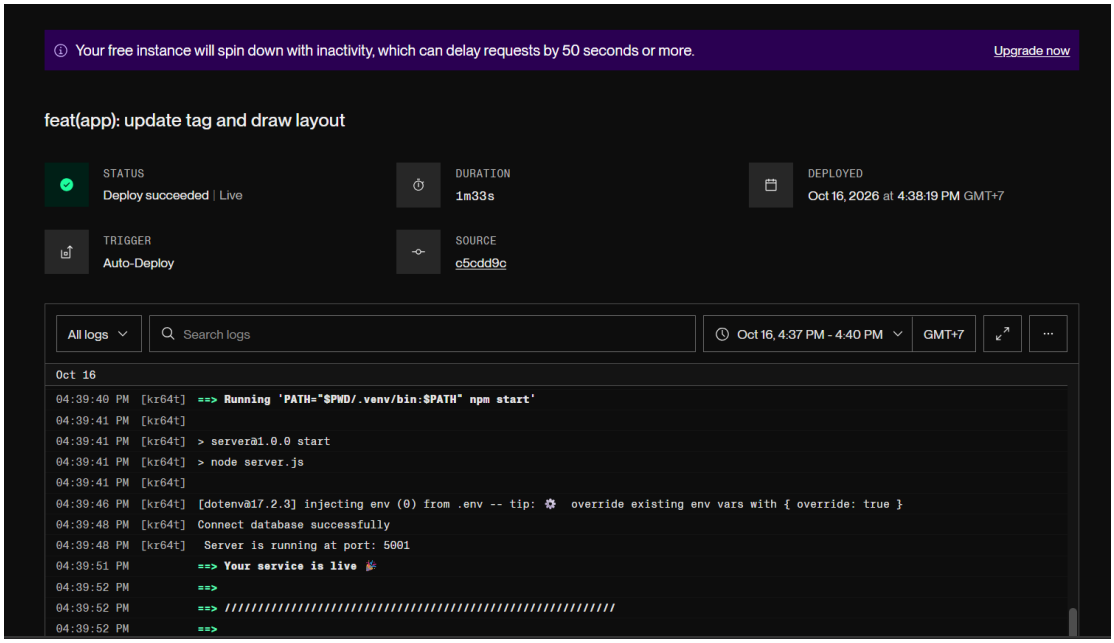
Hình 4.18. Giao diện lệnh build và start

Dashboard Environment ghi nhận các nhóm ACCESS_TOKEN_SECRET, CLIENT_ORIGINS,BREVO_API_KEY, CLOUDINARY_*, GOOGLE_CLIENT_ID, MONGODB_CONNECTIONSTRING và PORT. Chúng thuộc cấu hình backend; các giá trị trong bộ ảnh được che. Tên biến xác nhận điểm cấu hình, còn giá trị phù hợp và quyền truy cập phải được kiểm tra theo từng luồng.



Hình 4.19. Các nhóm biến môi trường backend trên Render

Log bổ sung cho deployment c5cdd9c hiển thị Deploy succeeded và Live, thời lượng 1 phút 33 giây, mốc ngày 16/10/2026. Tiến trình chạy node server.js, báo "Connect database successfully", mở cổng 5001 và thông báo service live.



Hình 4.20. Log backend kết nối dữ liệu và khởi động thành công

Kết quả khởi động cho thấy tiến trình đã đi qua bước kết nối dữ liệu trong lần chạy được chụp. Nó chưa bao quát quyền truy cập từng document hoặc độ ổn định kéo dài.

Commit backend c5cdd9c và commit frontend 214717b ở các ảnh được ghi riêng; hệ thống tách deployment nên không mặc định hai nền tảng cùng phục vụ một commit.

Render đồng thời cảnh báo instance Free có thể ngừng khi không hoạt động, khiến yêu cầu sau đó chậm 50 giây hoặc hơn. Đây là điều kiện của môi trường thực nghiệm cần tính đến khi đánh giá lần truy cập đầu. Bộ ảnh không cung cấp phép đo cold start có kiểm soát để định lượng độ trễ thực tế của Shrimple. [3]

4.7.1 Đối chiếu cấu hình build với tiến trình chạy

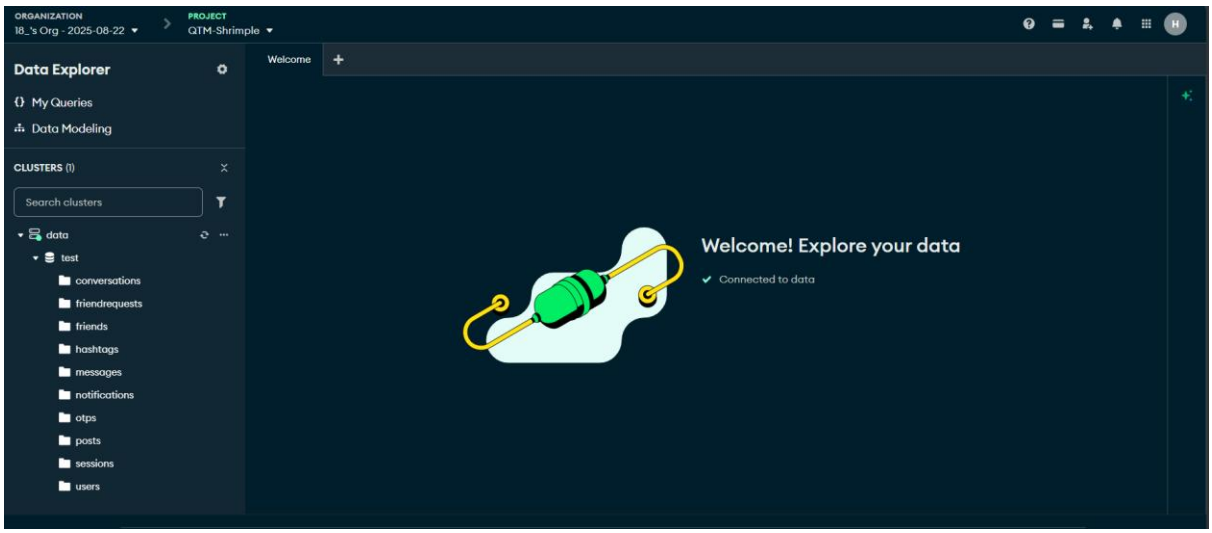
Frontend và backend có sản phẩm triển khai khác nhau. Vercel cần tập tài nguyên đầu ra của client; Render cần một tiến trình server tiếp tục chạy để nhận API và socket. Vì vậy, lệnh build frontend không thể dùng thay lệnh khởi động backend, còn Start Command của Render phải khởi chạy package server trong đúng thư mục làm việc. [10]

Trong cấu hình được ghi nhận, môi trường Python được tạo ở .venv trong quá trình build, sau đó thư mục thực thi được thêm vào PATH khi khởi động. Cách đọc này giải thích mối liên hệ giữa Build Command và Start Command. Tuy nhiên, việc cài thư viện Python chỉ xác nhận bước chuẩn bị dependency; kết quả của chức năng sử dụng các thư viện đó vẫn cần request và dữ liệu thử tương ứng. [10]

Để xác định backend sẵn sàng, cần đối chiếu cả trạng thái deployment, log tiến trình và một endpoint phù hợp. Log kết nối dữ liệu hỗ trợ kiểm tra bootstrap, còn phản hồi HTTP hỗ trợ kiểm tra đường truy cập bên ngoài. Nếu tiến trình khởi động lại hoặc mất kết nối, phải chọn log của lần chạy và khoảng thời gian đang có vấn đề, tránh dùng ảnh khởi động thành công trước đó làm kết luận cho sự cố hiện tại.

4.8 Cơ sở dữ liệu trên MongoDB Atlas

Ảnh Data Explorer thuộc project QTM-Shrimple hiển thị kết nối tới cluster data và cơ sở dữ liệu test. Danh sách collection gồm conversations, friendrequests, friends, hashtags, messages, notifications, otps, posts, sessions và users. Đây là cấu trúc dữ liệu hiển thị trong giao diện quản lý Atlas.



Hình 4.21. MongoDB Atlas và các collection của cơ sở dữ liệu test

Các tên collection phù hợp với nhóm chức năng của ứng dụng, nhưng ảnh không hiển thị nội dung document, schema, index hoặc quyền database user. Do đó, có thể xác nhận danh sách cấu trúc đang thấy và kết hợp với log kết nối backend; chưa kết luận mọi truy vấn, quan hệ tham chiếu hoặc phân quyền dữ liệu đã được kiểm thử. [4][10]

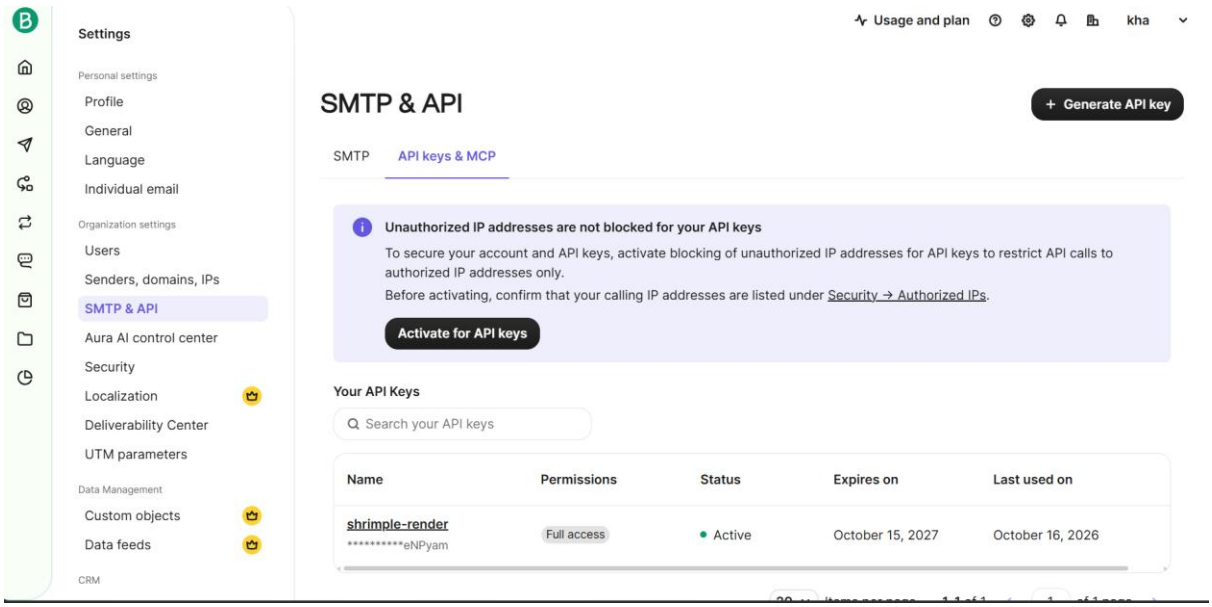
4.8.1 Trình tự khoanh vùng lỗi kết nối dữ liệu

Khi backend báo lỗi MongoDB, bước đầu là xác nhận tên biến được đọc và môi trường đang chạy. Tiếp theo kiểm tra khả năng phân giải địa chỉ, điều kiện TLS, quyền database user và phạm vi nguồn kết nối được phép. Các yếu tố cần đối chiếu với nội dung lỗi; không nên thay đồng thời nhiều cấu hình vì sẽ khó xác định điều kiện nào giải quyết vấn đề. [4][10]

Sau khi kết nối thành công, cần thử một thao tác dữ liệu đại diện bằng tài khoản được phép. Chẳng hạn, dữ liệu thử tạo qua ứng dụng phải xuất hiện lại đúng sau tải lại và không được truy cập từ tài khoản ngoài quyền. Kết nối ở lớp driver và tính đúng ở lớp nghiệp vụ là hai đối tượng đánh giá khác nhau; log hiện có hỗ trợ đối tượng thứ nhất, còn kịch bản CRUD bổ sung sẽ hỗ trợ đối tượng thứ hai.

4.9 Email giao dịch qua Brevo

Ảnh Brevo bổ sung hiển thị khóa có tên shrimple-render, quyền Full access và trạng thái Active. Giao diện ghi hạn dùng ngày 15/10/2027 và lần dùng gần nhất ngày 16/10/2026; giá trị khóa được che một phần.



Hình 4.22. Khóa API Brevo của tích hợp backend ở trạng thái Active

Trạng thái Active và mốc last used xác nhận thông tin sử dụng khóa theo Dashboard, nhưng không nhận diện một email OTP cụ thể. Để xác nhận luồng email, cần đối chiếu thao tác yêu cầu OTP, kết quả API hoặc nhật ký gửi và thư nhận trong hộp thư thử nghiệm. [5]

Thông báo trên màn hình cho biết chưa bật chặn IP không được phép cho API key. Đây là một điểm cấu hình cần rà soát cùng phạm vi quyền và địa chỉ gọi từ backend; ảnh không cho thấy một thao tác thay đổi chính sách đã được thực hiện.

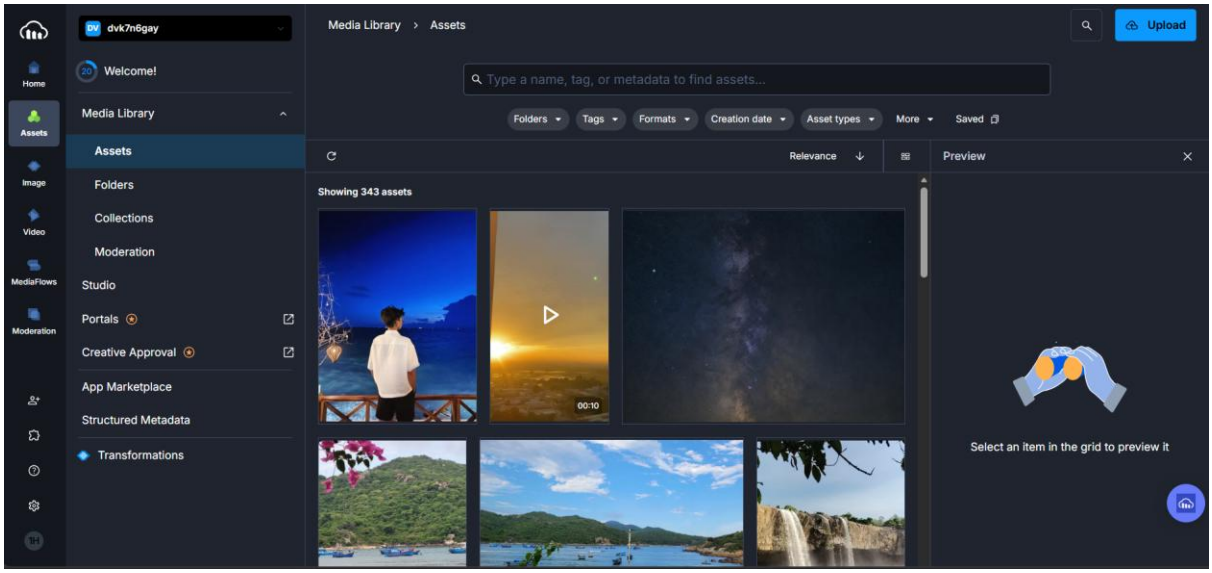
4.9.1 Đối chiếu cấu hình email với một lần yêu cầu OTP

Để kiểm tra tích hợp, nhóm cần xác định backend đang dùng nhánh gửi Brevo HTTP API theo cấu hình, thông tin người gửi phù hợp và khóa có quyền gọi cần thiết. Tên EMAIL_USER hoặc EMAIL_PASS trên Dashboard không tự xác định nhánh vận chuyển đang hoạt động vì source có nhiều cách gửi. Phải liên hệ cấu hình với logic chọn nhánh và kết quả của lần gọi tương ứng. [10]

Một lần thử nên dùng email kiểm tra do nhóm quản lý và ghi lại thời điểm yêu cầu. Kết quả từ backend được đối chiếu với sự kiện gửi ở Brevo và thư trong hộp thư; nếu có độ trễ, ghi thời điểm ở từng bước. Khi thất bại, cần phân biệt lỗi gọi API, lỗi người gửi hoặc phát thư với lỗi nhập mã ở ứng dụng. Các trường hợp này thuộc các bước khác nhau nên cần bằng chứng tương ứng. [5]

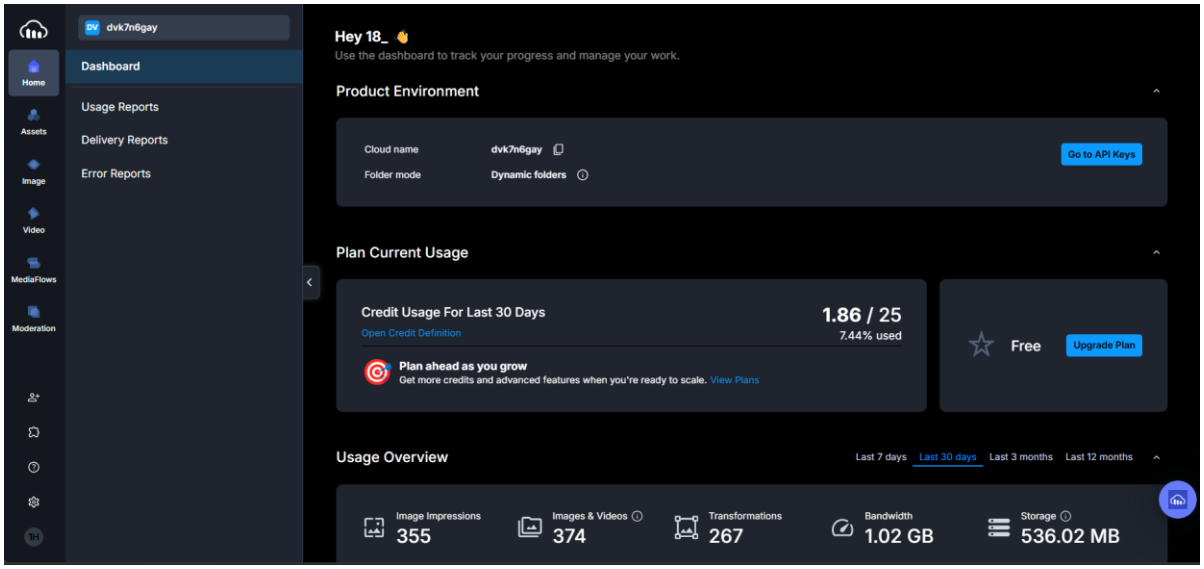
4.10 Tài nguyên media trên Cloudinary

Media Library của Cloudinary hiển thị 343 assets cùng ảnh và video trong giao diện quản lý. Để gắn một tài nguyên cụ thể với thao tác upload của Shrimple, cần đối chiếu public ID hoặc URL và kết quả trả về từ API ứng dụng. [6]



Hình 4.23. Thư viện ảnh và video trong Cloudinary

Dashboard của cloud dvk7n6gay hiển thị gói Free và mức sử dụng credit 1,86/25, tương đương 7,44% trong 30 ngày. Khu vực Usage Overview còn ghi bandwidth 1,02 GB và storage 536,02 MB. Các giá trị là ảnh chụp mức sử dụng tài khoản trong khung lọc hiển thị, không phải số liệu đo tốc độ của ứng dụng.



Hình 4.24. Mức sử dụng tài nguyên Cloudinary trong khung thời gian hiển thị

Chỉ tiêu Images & Videos trong Usage Overview và số assets ở Media Library có phạm vi hiển thị khác nhau; báo cáo không đồng nhất hai bộ đếm. Cấu hình CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY và

CLOUDINARY_API_SECRET thuộc backend. Cần tiếp tục kiểm tra loại tệp, dung lượng và quyền upload theo nghiệp vụ. [10]

4.10.1 Đối chiếu tài nguyên và xử lý các trường hợp không hoàn tất

Khi kiểm tra upload, một URL có thể tải được chưa đủ xác định quyền tạo hoặc sửa nội dung. Cần đối chiếu tài khoản thực hiện, tệp thử, phản hồi backend và mục nội dung được gắn tài nguyên. Sau tải lại, liên kết cần tiếp tục đúng theo thiết kế; khi xóa nội dung, nhóm cần xác định rõ tài nguyên media được giữ hay xóa để tránh kỳ vọng không thống nhất.

Các bước upload và ghi dữ liệu có thể không cùng hoàn tất trong một lần lỗi. Vì vậy, kế hoạch thử nên có tình huống dịch vụ media lỗi hoặc thao tác lưu nội dung không thành công. Mục tiêu là ghi nhận trạng thái thực tế, phản hồi cho người dùng và cách xử lý tài nguyên chưa gắn; báo cáo chưa xác nhận cơ chế dọn dẹp tự động hiện có. [10]

4.11 Thiết kế kịch bản kiểm thử và cách ghi nhận

Để chuyển từ quan sát thành nghiệm thu, mỗi ca kiểm thử cần có mã nhận diện, mục tiêu, điều kiện đầu vào, thao tác và kết quả mong đợi. Kết quả thực tế phải được ghi sau khi thực hiện, kèm thời điểm và phiên bản. Bộ kịch bản dưới đây là kế hoạch kiểm tra tiếp theo; các ô mô tả điều kiện đạt không được xem là kết quả đã đạt.

Môi trường kiểm tra cần có ít nhất hai tài khoản thử nghiệm cho các luồng tương tác. Hai phiên trình duyệt nên được phân biệt rõ để tránh nhầm trạng thái đăng nhập. Khi khảo sát quyền, cần tạo tình huống chủ sở hữu, thành viên được phép và người ngoài phạm vi tài nguyên. Dữ liệu thử phải nhận diện được để đối chiếu sau tải lại và dọn dẹp khi hoàn tất.

Bảng 4.4. Kế hoạch kiểm thử để bổ sung nghiệm thu chức năng

Mã	Kịch bản	Điều kiện đạt cần kiểm tra
TC01	Truy cập trực tiếp /signup và /forgot-password	Trang đúng tiếp tục hiển thị sau tải lại; tài nguyên không lỗi
TC02	GET /api/users/me khi chưa đăng nhập	Nhận mã và JSON theo hợp đồng; không nhận HTML fallback

Mã	Kịch bản	Điều kiện đạt cần kiểm tra
TC03	Đăng nhập và làm mới phiên	Người dùng hợp lệ truy cập đúng dữ liệu; trạng thái sau tải lại nhất quán
TC04	Đăng xuất rồi gọi endpoint được bảo vệ	Phiên cũ không tiếp tục cấp quyền vượt điều kiện thiết kế
TC05	Yêu cầu OTP và nhận thư thử	Đối chiếu một yêu cầu với nhật ký gửi và thư đến hộp thư
TC06	Nhập OTP sai hoặc hết hạn	Backend từ chối bước nghiệp vụ tiếp theo; lỗi được hiển thị phù hợp
TC07	Hai tài khoản gửi và nhận tin	Hai đầu hiển thị nội dung đúng; tin tồn tại sau tải lại theo thiết kế
TC08	Người không có quyền truy cập hội thoại	API và sự kiện liên quan không cung cấp nội dung ngoài quyền
TC09	Mất mạng và kết nối lại khi chat	Nhận biết trạng thái; đồng bộ lại và kiểm tra mất hoặc trùng nội dung
TC10	Upload tệp hợp lệ	Có kết quả API và URL đối chiếu được với Cloudinary và giao diện
TC11	Upload loại tệp hoặc dung lượng không phù hợp	Đầu vào bị xử lý theo quy tắc; trạng thái lỗi không tạo nội dung sai
TC12	Kết nối từ origin Preview	CORS và socket cho phép hoặc từ chối đúng chính sách đã xác định
TC13	Đối chiếu CRUD dữ liệu thử	Thay đổi qua API phản ánh đúng trạng thái, quan hệ và quyền
TC14	So sánh lần truy cập đầu với dịch vụ đã hoạt động	Ghi điều kiện và thời gian riêng; không trộn cold start vào một số trung bình
TC15	Cập nhật một biến kết nối ở môi trường thử	Deployment mới dùng đúng đích; các luồng còn lại được đối chiếu riêng

Với mỗi ca, bằng chứng nên ngắn nhưng đủ nối đầu vào với kết quả: ảnh hai đầu khi cần, mã HTTP, định danh dữ liệu thử hoặc log có thời gian. Các lần thử lỗi cần ghi cả kết quả thực tế và khác biệt so với mong đợi. Một ảnh báo thành công đơn lẻ khó giải thích nguyên nhân nếu lần kiểm tra sau cho kết quả khác.

4.11.1 Phương pháp đo hiệu năng trong giai đoạn tiếp theo

Đo trải nghiệm cần xác định thiết bị, trình duyệt, điều kiện mạng, vị trí và trạng thái cache. Các lần đo lặp lại nên cùng điều kiện, đồng thời tách tải trang lần đầu với truy cập đã có tài nguyên trong cache. Đối với Render Free, cần ghi trạng thái dịch vụ trước phép đo để phân biệt cold start với xử lý request khi tiến trình đã hoạt động. [3]

Đo tải backend cần xác định nghiệp vụ, số người dùng hoặc kết nối đồng thời, tần suất request và tiêu chí lỗi chấp nhận. Báo cáo kết quả nên có số mẫu và phân bố thời gian thay vì chỉ một giá trị nhanh nhất. Khi chưa có bộ đo này, các số build time, bandwidth và credit chỉ được dùng để mô tả build hoặc sử dụng dịch vụ, không đại diện khả năng chịu tải.

4.12 Tổng hợp mức độ kiểm chứng

Bảng 4.5. Mức độ kiểm chứng sau khi bổ sung ảnh dịch vụ

Hạng mục	Bằng chứng hiện có	Giới hạn cần giữ
Frontend Vercel	Ready, domain hợp lệ, log build và commit	Chưa tương đương nghiệm thu toàn bộ ứng dụng
Routing và HTTP	Trang công khai trả 200 HTML; /api/users/me trả 401 JSON	Phép thử chưa xác thực, thuộc ngày 16/10/2026
Giao diện nội bộ	Ảnh bảng tin và chat nhóm có nội dung	Chưa có biên bản thao tác hai đầu
Backend và dữ liệu	Render Live; log kết nối; Atlas có 10 collection hiển thị	Chưa có phép thử CRUD và phân quyền từng nghiệp vụ
Email	Brevo key Active, có mốc last used	Chưa có cặp nhật ký gửi và hộp thư nhận OTP
Media	Cloudinary có assets và số liệu sử dụng	Chưa có chuỗi upload → URL → hiển thị được đối chiếu
Hiệu năng và tính năng nâng cao	Có chỉ tiêu build và dependency liên quan	Chưa có đo tải, độ trễ, cuộc gọi, camera hoặc AI

CHƯƠNG 5: ĐÁNH GIÁ KẾT QUẢ VÀ KẾT LUẬN

5.1 Kết quả đạt được

Đề tài làm rõ chuỗi triển khai frontend React và Vite từ GitHub đến Vercel, trong đó client là thư mục build và dist là đầu ra. Cấu hình rewrite phân biệt URL giao diện với REST API; các biến frontend, socket và backend được phân tách theo nơi sử dụng. Các bằng chứng về domain, commit và log cho phép đối chiếu một bản triển khai cụ thể. [10]

Ở lớp truy cập công khai, bộ quan sát ghi nhận giao diện tải được, URL con có HTML và endpoint được bảo vệ trả 401 JSON khi chưa xác thực. Bộ ảnh bổ sung mở rộng minh chứng sang frontend có nội dung, backend Live, kết nối dữ liệu, các collection Atlas, trạng thái khóa Brevo và tài nguyên Cloudinary. Những kết quả này xác lập các thành phần có trong mô hình và trạng thái nhìn thấy ở từng mốc.

Từ góc nhìn Quản trị mạng, giá trị của mô hình là khả năng xác định trách nhiệm theo đường kết nối: Vercel phân phối giao diện và có thể proxy REST; Render xử lý nghiệp vụ; dịch vụ dữ liệu, email và media có điều kiện vận hành riêng. Vì vậy, chẩn đoán sự cố cần bắt đầu từ loại request và phản hồi cụ thể, sau đó mới chọn cấu hình hoặc log để kiểm tra.

5.2 Phân tích kết quả theo các lớp vận hành

Ở lớp xuất bản, Ready, log hoàn tất và danh sách tài nguyên tạo thành chuỗi bằng chứng về bản build đã được Vercel phát hành. Ở lớp truy cập, ảnh giao diện và kết quả HTTP bổ trợ cho nhau: status xác định phản hồi của máy chủ, còn ảnh cho biết React đã dựng được nội dung. Việc URL con hoạt động sau tải lại hỗ trợ đánh giá fallback SPA trong phạm vi được thử.

Ở lớp dịch vụ, Render Live và log kết nối dữ liệu cho thấy một lần khởi động hoàn tất; Atlas, Brevo và Cloudinary cung cấp các trạng thái tương ứng tại giao diện quản lý. Tuy nhiên, quan hệ giữa các lớp chỉ được xác nhận đầy đủ khi cùng một thao tác nghiệp vụ được đối chiếu xuyên suốt. Đây là điểm khác nhau giữa kiểm tra triển khai từng thành phần và kiểm tra đầu cuối toàn hệ thống.

Ở lớp quản trị, các mốc commit và thời gian giúp tránh so sánh những trạng thái không cùng phiên bản. Phân tách dữ liệu quan sát với quy trình đề xuất cũng giữ cho kết luận có thể kiểm tra lại. Một quy trình quản lý tốt cần lưu thông tin đủ để người khác biết phải mở deployment nào, gọi URL nào và kỳ vọng phản hồi trong điều kiện nào.

5.3 Hạn chế và các vấn đề cần hoàn thiện

Bộ kết quả chưa có kịch bản đầu cuối được ghi đầy đủ cho OTP, đăng nhập và làm mới phiên, phân quyền nhóm, gửi nhận realtime hoặc upload media. Các ảnh hiển thị giao diện và trạng thái dịch vụ bổ trợ cho phân tích, nhưng chưa cung cấp toàn bộ dữ liệu để nghiệm thu các luồng này. Chức năng cuộc gọi, camera và dịch thuật cần thiết bị, người tham gia cùng dịch vụ AI tương ứng.

Về vận hành, chưa có kiểm thử tải, đo độ trễ theo vùng hay dữ liệu Core Web Vitals thực của Shrimple. Instance Render Free có giới hạn khởi động lại sau thời gian không hoạt động; số liệu sử dụng Cloudinary chỉ phản ánh tài khoản trong khung thời gian hiển thị. Cấu hình Production Overrides và các commit khác nhau giữa Vercel, Render cần được ghi lại khi tái hiện sự cố. [3]

5.4 Định hướng cải tiến và quy trình xử lý sự cố

Bảng 5.1. Định hướng khoanh vùng sự cố theo từng lớp

Dấu hiệu	Điểm kiểm tra ưu tiên	Hướng xử lý
Ready nhưng trang trắng	Root Directory, dist, index.html, lỗi JavaScript	Đối chiếu output và request tài nguyên của đúng deployment
URL con trả 404	Fallback SPA và đường dẫn React Router	Kiểm tra mở trực tiếp và tải lại URL
API trả HTML	Thứ tự rewrite và URL request	Đặt tuyến /api đúng vị trí trước fallback
API trả 401 hoặc 403	Phiên đăng nhập và quyền endpoint	Phân biệt thiếu xác thực với thiếu quyền nghiệp vụ
Socket không kết nối	VITE_SOCKET_URL, origin, token, Render	Kiểm tra handshake và kết nối lại riêng với REST
Biến mới chưa có hiệu lực	Môi trường, commit và thời điểm build	Tạo deployment dùng biến mới rồi kiểm tra lại

Dấu hiệu	Điểm kiểm tra ưu tiên	Hướng xử lý
Lần truy cập đầu chậm	Trạng thái instance và log backend	Đo riêng cold start và yêu cầu khi dịch vụ đã hoạt động

Ưu tiên tiếp theo là xây dựng một bộ nghiệm thu bằng tài khoản thử nghiệm. Mỗi kịch bản cần ghi môi trường, phiên bản frontend và backend, điều kiện đầu vào, thao tác, kết quả mong đợi và kết quả thực tế. Các ca OTP phải bao gồm mã sai hoặc hết hạn; chat cần hai người dùng và mất kết nối; upload cần đối chiếu tệp, URL trả về và quyền truy cập.

Sau khi các luồng chính có kết quả lặp lại được, nhóm có thể bổ sung kiểm tra tự động URL công khai, giám sát lỗi theo lớp và đo trải nghiệm tải trang. Quy trình Preview trước Production, rà soát cache của API cá nhân và diễn tập khôi phục trong môi trường phù hợp giúp giảm rủi ro khi cập nhật. [1]

5.5 Quản lý rủi ro cấu hình và dịch vụ

Hệ thống sử dụng nhiều nhà cung cấp nên rủi ro không chỉ nằm ở mã nguồn. Một bản frontend mới có thể gọi sai backend, một origin Preview có thể chưa được cho phép, hoặc khóa dịch vụ đã đổi nhưng tiến trình vẫn dùng cấu hình cũ. Để xử lý, nên quản lý thay đổi theo thành phần chịu ảnh hưởng và kiểm tra kết nối sau khi phát hành, thay vì thay nhiều giá trị cùng lúc rồi chỉ nhìn trang đầu.

Bảng 5.2. Rủi ro vận hành và biện pháp kiểm soát đề xuất

Rủi ro	Biểu hiện có thể gặp	Biện pháp đề xuất
Sai phiên bản hoặc môi trường	Kết quả khác giữa domain chính và URL bản thử	Ghi commit, môi trường và URL cho mỗi lần kiểm tra
Sai địa chỉ hoặc origin	REST hoạt động nhưng socket thất bại	Đối chiếu URL và chính sách từng kênh kết nối
Khóa dịch vụ quá rộng hoặc không còn phù hợp	Tích hợp mất quyền hoặc sử dụng ngoài dự kiến	Rà soát phạm vi quyền, nguồn gọi và vòng đời thay khóa
Gián đoạn backend	Truy cập đầu chậm hoặc kết nối bị ngắt	Theo dõi trạng thái instance; thử reconnect và phân loại cold start

Rủi ro	Biểu hiện có thể gặp	Biện pháp đề xuất
Không nhất quán dữ liệu và media	Nội dung tham chiếu URL không còn dùng được	Đối chiếu dữ liệu với tài nguyên; xác định quy tắc xóa và dọn dẹp
Thiếu dữ liệu điều tra	Có lỗi nhưng khó tái hiện	Lưu thời điểm, phiên bản, request và log đúng phạm vi

5.5.1 Quy trình phản ứng khi có sự cố

Bước đầu là mô tả biểu hiện bằng URL, thời điểm và thao tác, đồng thời xác định lỗi ảnh hưởng một người dùng hay nhiều người. Sau đó kiểm tra frontend có nhận HTML và bundle không. Nếu tài nguyên đã tải, tiếp tục phân loại request REST, socket hoặc media; đọc mã HTTP và nội dung lỗi trước khi chuyển sang log của backend hay dịch vụ ngoài.

Khi lỗi xuất hiện sau cập nhật, so sánh thay đổi của đúng commit và cấu hình triển khai. Nếu lựa chọn khôi phục frontend, cần xác định bản đủ điều kiện và kiểm tra tương thích backend. Sau khôi phục vẫn thử lại URL, API và chức năng chịu ảnh hưởng; trạng thái domain đổi về bản cũ chỉ xác nhận bước chuyển bản, chưa xác nhận mọi tác động đã được khắc phục. [1]

Với sự cố dữ liệu, email hoặc media, phạm vi khôi phục khác với frontend. Dữ liệu đã ghi và thư đã phát cần cách xử lý nghiệp vụ riêng; backup dữ liệu cũng cần kiểm tra khả năng khôi phục và quyền truy cập. Hồ sơ sau xử lý nên ghi nguyên nhân có bằng chứng, thay đổi đã áp dụng và kết quả kiểm tra lại để tránh lặp một biện pháp không liên quan ở lần sau.

5.6 Thứ tự ưu tiên hoàn thiện

Ưu tiên đầu tiên là hoàn tất các luồng lõi có người dùng thử: xác thực, OTP, chat, phân quyền và media. Tiếp theo là kiểm tra gián đoạn mạng, cấu hình môi trường và các tình huống lỗi dịch vụ. Khi có kết quả nền tảng, nhóm mới có cơ sở đo tải, tách bundle hoặc điều chỉnh gói tài nguyên theo nhu cầu quan sát được.

Các chức năng cuộc gọi, camera và AI nên có kế hoạch riêng vì phụ thuộc thiết bị và điều kiện xử lý khác với tải trang. Mỗi phần mở rộng cần xác định giá trị sử dụng,

môi trường tối thiểu và tiêu chí đạt. Cách ưu tiên này giữ phạm vi báo cáo Vercel rõ ràng, đồng thời tạo lộ trình tiếp tục phát triển toàn hệ thống.

5.7 Kết luận

Vercel đáp ứng vai trò xuất bản frontend của Shrimple trong mô hình đã khảo sát, đồng thời cho phép gắn bản build với mã nguồn, môi trường và domain. Kết quả hiện có xác nhận những bước triển khai và trạng thái dịch vụ được minh họa; mức sẵn sàng của toàn hệ thống cần được kết luận sau khi hoàn tất kiểm thử đầu cuối và đo lường vận hành. Cách đánh giá theo lớp giúp báo cáo kết nối được cấu hình, bằng chứng và giới hạn của từng nhận định.

BẢNG PHÂN CHIA CÔNG VIỆC

Quách Hoàng Kha phụ trách chính các hạng mục kỹ thuật phức tạp và tổng hợp báo cáo. Các thành viên còn lại đảm nhận những phần việc cụ thể như sau.

Bảng 5.3. Phân công nhiệm vụ của các thành viên

Mã số sinh viên	Họ và tên	Nhiệm vụ
3122410167	Quách Hoàng Kha	Chủ trì thiết kế kiến trúc; cấu hình và triển khai frontend Vercel, backend Render; xử lý rewrite /api, SPA fallback, biến môi trường, CORS và Socket.IO; tích hợp MongoDB Atlas, Brevo, Cloudinary; phân tích log và xử lý lỗi triển khai; viết nội dung kỹ thuật trọng tâm Chương 3–4, tổng hợp và rà soát báo cáo.
3124410183	Tô Hoàng Nhật Lâm	Phân tích cấu trúc backend, API và cơ sở dữ liệu; mô tả các collection MongoDB Atlas; đối chiếu cấu hình Render và log kết nối; phối hợp viết phần backend và dữ liệu trong Chương 3–4.
3124410308	Nguyễn Phúc Tài	Khảo sát giao diện và các luồng đăng nhập, đăng ký, khôi phục mật khẩu, bảng tin và chat; xây dựng kịch bản kiểm thử HTTP, xác thực, phân quyền và realtime; tổng hợp kết quả quan sát ở Chương 4.
3124560062	Trần Quỳnh Ngọc	Biên soạn phần giới thiệu và cơ sở lý thuyết Chương 1–2; tìm và chọn tài liệu tham khảo cần thiết; chuẩn hóa thuật ngữ, trích dẫn và kiểm tra nội dung lý thuyết.
3124560100	Nguyễn Đinh Ý Vy	Tổng hợp phần đánh giá, hạn chế và hướng phát triển ở Chương 5; sắp xếp ảnh minh chứng, chú thích hình và bảng; rà soát chính tả, mục lục và định dạng báo cáo.

TÀI LIỆU THAM KHẢO

[1] Vercel. Vercel Documentation: triển khai Vite, cấu hình build, Git, môi trường, định tuyến, HTTPS, giám sát và khôi phục phiên bản. https://vercel.com/docs

[2] Vite. Env Variables and Modes. https://vite.dev/guide/env-and-mode

[3] Render. Render Documentation: triển khai Node.js/Express, WebSocket và giới hạn dịch vụ miễn phí. https://render.com/docs

[4] MongoDB. MongoDB Documentation: mô hình dữ liệu và kết nối MongoDB Atlas. https://www.mongodb.com/docs/

[5] Brevo. Send a Transactional Email. https://developers.brevo.com/docs/send-a-transactional-email

[6] Cloudinary. Node.js SDK: tích hợp và tải ảnh, video. https://cloudinary.com/documentation/node_integration

[7] MDN Web Docs. Cross-Origin Resource Sharing (CORS). https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS

[8] npm Docs. npm CLI: các lệnh npm ci và npm audit. https://docs.npmjs.com/cli/v11/commands/

[9] Socket.IO. Documentation v4: Offline Behavior và Connection State Recovery. https://socket.io/docs/v4/

[10] Nhóm thực hiện. QtmShrimple: mã nguồn và cấu hình hệ thống https://github.com/HuangKha097/QtmShrimple

[11] Nhóm thực hiện. QtmShrimple: video demo quy trình triển khai (deploy) hệ thống. https://youtu.be/a3tnFGcjhHc?si=mw6SNl7CUTtz7ssD